

CPSC 250
Programming for Data Manipulation
Complete Lecture Notes

Edward Brash

Contents

Week 1: Python Review and Core Data Structures	3
0.1 Primitive Data Types	4
0.2 Lists, Tuples, and Sets	11
0.3 Dictionaries	16
0.4 Branching and Flow Control	20
0.5 Functions	25
0.6 Big Program One	30
0.7 Big Program Two	43
 Week 2: Object-Oriented Programming	 55
0.8 Philosophy of Complex Objects	56
0.9 Classes and Objects	60
0.10 Class Constructors	65
0.11 Class Customization	69
0.12 Operator Overloading	75
0.13 Memory Allocation	79
0.14 Classes Program Example	86
0.15 Airline Reservation System	91
0.16 Craps Game Odds	96
0.17 Plotting Credits	101
 Week 3: Inheritance, Recursion, Algorithms, and Testing	 105
0.18 Derived Classes	106
0.19 Inheritance and Polymorphism	110
0.20 Multiple Inheritance and Mixins	114
0.21 Plant Information Example	118
0.22 Fibonacci	123
0.23 Complex Algorithms	128
0.24 Unit Tests and Timers	134
 Week 4: Plotting, Regression, and Data Analysis	 141
0.25 More on Fibonacci	142
0.26 Basic Plotting of Data	149
0.27 StatsModels Linear Regression	155

0.28 Plotting Error Bands	159
0.29 Advanced Linear Regression	163

Week 1: Python Review and Core Data Structures

CPSC 250

Lecture 1: Primitive Data Types and Memory

Edward Brash

1 Introduction

A common misconception among students entering CPSC 250 is that they should already remember every detail from CPSC 150/150L perfectly. In reality, students enter with a wide range of preparation levels.

The goal of this course is not only to reinforce earlier material, but also to develop a much deeper understanding of how Python actually works internally.

In particular, we will focus heavily on:

- Data structures
- Memory representation
- Algorithmic thinking
- Object-oriented programming
- Numerical data representation

2 Why Data Structures Matter

In many ways, data structures are the central idea in programming.

Every piece of data in a computer program must ultimately be stored somewhere in memory. Understanding:

- how data is represented,
- how it is organized,
- and how efficiently it can be manipulated

is essential for designing good algorithms.

Python intentionally hides many low-level implementation details from programmers. This is convenient, but it can also obscure what is really happening underneath the hood.

In this course, we will repeatedly pull back that abstraction layer.

3 Primitive Data Types in Python

The four basic primitive-style data types we will review are:

1. Integers
2. Floating-point numbers
3. Strings
4. Booleans

3.1 Examples

Type	Examples
Integer	1, 0, 42, -6
Float	1.2, -3.14159, 3.28e-6
String	"a", "Bob", \n
Boolean	True, False

3.2 Important Notes

Different data types:

- require different amounts of memory,
- use different storage mechanisms,
- and support different operations.

4 Integers in Python

One of Python's nicest features is that integers effectively behave as arbitrary-precision numbers.

Unlike C or C++, Python integers do not normally overflow when they exceed a fixed size.

4.1 Overflow in C/C++

In low-level languages like C/C++, integer variables are allocated a fixed number of bytes.

For example:

- 4-byte integers can store only a limited range of values.
- Exceeding that range produces overflow.

Python hides this complexity from the programmer.

5 Python Integers are Objects

Although integers appear to be primitive values, Python integers are actually full Python objects implemented internally in C.

Conceptually:

- a Python integer stores metadata,
- bookkeeping information,
- and dynamically allocated memory.

This flexibility comes at the cost of additional memory overhead.

5.1 Factorial Example

```
1 def factorial(n):  
2     if n == 0 or n == 1:  
3         return 1  
4  
5     return n * factorial(n - 1)  
6  
7 print(factorial(231))
```

Python can compute extremely large integers with hundreds of digits automatically. Internally, Python allocates additional memory as necessary.

5.2 Memory Cost

Even very small integers require substantial overhead in Python.

A Python integer may require roughly 28 bytes or more of memory, depending on the system architecture.

This is because Python integers behave more like objects than primitive machine-level integers.

6 Floating Point Numbers

Most programming languages represent floating point numbers using binary fractions.

This introduces important limitations:

- (a) Some decimal numbers cannot be represented exactly.
- (b) Floating-point arithmetic introduces rounding error.

7 Scientific Notation Review

Recall ordinary scientific notation:

$$3352.28 = 3.35228 \times 10^3$$

This representation separates a number into:

- mantissa/significand
- base
- exponent

8 Binary Fractions

Decimal fractions use powers of 10:

$$32.56 = 3(10^1) + 2(10^0) + 5(10^{-1}) + 6(10^{-2})$$

Binary fractions instead use powers of 2.

For example:

$$1011.011_2$$

corresponds to:

$$1(2^3) + 0(2^2) + 1(2^1) + 1(2^0) + 0(2^{-1}) + 1(2^{-2}) + 1(2^{-3})$$

which equals:

$$8 + 0 + 2 + 1 + 0 + 0.25 + 0.125 = 11.375$$

9 Binary Scientific Notation

Floating point numbers are stored in normalized binary scientific notation.

Example:

$$11.01_2 = 1.101_2 \times 2^1$$

The normalized form always begins with:

$$1.xxxxx$$

This allows one bit to be implicitly assumed rather than explicitly stored.

10 32-Bit Floating Point Representation

IEEE-754 single precision floating point numbers are stored using:

- 1 sign bit
- 8 exponent bits
- 23 mantissa bits

10.1 Exponent Bias

The exponent field uses a bias of 127.

Thus:

$$\text{stored exponent} = E + 127$$

Examples:

$$00000001_2 \rightarrow 2^{-126}$$

$$11111110_2 \rightarrow 2^{127}$$

Special exponent values are reserved for:

- infinity
- NaN (Not a Number)
- denormalized numbers

11 Special Exponent Values

- exponent bits are all 1,
- and the mantissa is all 0,

then the number represents:

$$+\infty$$

or:

$$-\infty$$

depending on the sign bit.

Examples that may generate infinity include:

1 1.0 / 0.0

or arithmetic overflow.

11.0.1 NaN (Not a Number)

If:

- exponent bits are all 1,
- but the mantissa is not all 0,

then the result is NaN.

NaN represents undefined or invalid numerical operations.

Examples include:

```
1 0.0 / 0.0
2 math.sqrt(-1)
```

NaN has unusual comparison behavior:

```
1 float('nan') == float('nan')
```

returns:

```
1 False
```

This behavior is intentional and follows the IEEE-754 standard.

11.1 Summary

Exponent Field	Interpretation
00000000	Denormalized Numbers / Zero
00000001 to 11111110	Normal Floating Point Numbers
11111111	Infinity or NaN

12 Strings in Python

Strings are significantly more complicated internally than they first appear.

Consider:

```
1 s1 = "abcdef"
```

Conceptually, the characters are stored sequentially in memory:

a
b
c
d
e
f

Each character is represented internally using character encoding schemes such as ASCII or Unicode.

12.1 Memory Addresses

```
1 print(s1)
2 print(id(s1))
```

The function `id()` returns the memory identity of the object.

13 Booleans

In Python, booleans are actually a subclass of integers.

Specifically:

`True = 1`

and

`False = 0`

Internally, booleans therefore behave much like integers.

14 Key Takeaways

- Python hides many implementation details.
- Data representation strongly affects algorithm behavior.
- Integers, floats, strings, and booleans are all implemented as objects.
- Floating point numbers are approximations.
- Understanding memory representation is essential for serious programming.

CPSC 250

Lists, Tuples, and Sets

Edward Brash

1 Lists

Python lists are complex data structures that store a sequence of information. Lists usually contain related information, but they can contain multiple data types, including other lists.

```
1 grades = [52, 78, 100, 92, 32]
```

Each element in the list has an index.

Index	Value
0	52
1	78
2	100
3	92
4	32

1.1 Iterating Through a List

```
1 for i in range(len(grades)):
2     print(i, grades[i], id(grades[i]))
```

This prints the index, the value stored at that index, and the memory identity of the object.

1.2 A Conceptual Memory Picture

It is useful to imagine a list as a sequence of memory locations:

Index	Value	Conceptual Memory Location
0	52	31
1	78	32
2	100	33
3	92	34
4	32	35

However, this picture is only conceptual. Python does a lot of things under the hood to optimize list storage. The actual storage should not necessarily be interpreted as a simple sequential block of values.

1.3 Lists Can Store Mixed Information

```
1 employee_info = [  
2     "John",  
3     "Smith",  
4     1289,  
5     "Promenade Lane",  
6     229724182  
7 ]
```

This list contains a first name, last name, street number, street name, and Social Security number. Python permits this kind of mixed storage, although in larger programs we should be careful about whether this is the cleanest design.

2 Python List Methods

Some common list operations include:

- `append(value)`: add a value to the end of the list.
- `insert(index, value)`: insert a value at a particular position.
- `remove(value)`: remove the first occurrence of a value.
- `pop()`: remove and return the last element.
- `sort()`: sort the list in place.
- `reverse()`: reverse the list in place.

```
1 nums = [3, 1, 4]  
2 nums.append(10)  
3 nums.sort()  
4 print(nums)
```

3 Tuples

Tuples are sequence-like structures that cannot be changed once created. We say that tuples are **immutable**.

List	[1, 2, 3]
Tuple	(1, 2, 3)

Examples:

```
1 my_list = [1, 2, 3]  
2 my_tuple = (1, 2, 3)
```

Tuples are useful for data that should not change. They are often used for constants or for returning multiple values from a function.

Honestly, tuples are not always the first structure students reach for, but their immutability can be very useful.

4 Named Tuples

One problem with lists and ordinary tuples is that referring to elements by index can make code hard to read.

```
1 car = ('Mercedes', 'E350', 62000)
2 print(car[2])
```

The expression `car[2]` works, but it is not especially descriptive. A reader has to remember that index 2 means the price.

Python's `collections` package provides `namedtuple`, which allows us to refer to tuple elements by name.

```
1 from collections import namedtuple
2
3 Car = namedtuple('Car', ['make', 'model', 'price'])
4
5 mercedes = Car('Mercedes', 'E350', 62000)
6 bmw = Car('BMW', '325i', 67000)
7
8 print(mercedes.price)
9 print(bmw.model)
```

This produces code that is easier to read and less dependent on remembering numerical positions.

5 Sets

Sets in Python are unordered collections of unique objects. They behave much like sets in mathematics.

```
1 my_set = {1, 2, 3, 17}
```

Sets use curly braces:

{ }

5.1 List, Tuple, and Set Syntax

Structure	Syntax	Mutable?
List	[]	Yes
Tuple	()	No
Set	{ }	Yes

6 Using Sets to Remove Duplicates

Sets automatically eliminate duplicate values.

```
1 first_names = [
2     'bob',
3     'alice',
4     'jane',
5     'bob',
6     'fred',
```

```

7     'alice'
8 ]
9
10 first_names_set = set(first_names)
11
12 print(first_names_set)

```

Possible output:

```

1 {'bob', 'alice', 'jane', 'fred'}

```

The order of the output is not guaranteed, because sets are unordered.

7 Set Methods

Useful set methods include:

- `len(my_set)` — return the number of elements.
- `set1.update(set2)` — add all elements from another set.
- `my_set.add(value)` — add one value.
- `my_set.remove(value)` — remove a value.
- `my_set.pop()` — remove and return an arbitrary value.
- `my_set.clear()` — remove all values.

7.1 Important Warning About `pop()`

For lists, `pop()` removes the last element by default. For sets, however, there is no “last” element because sets are unordered.

Therefore:

```

1 my_set.pop()

```

removes an arbitrary element. You should not write code that depends on which element will be removed.

8 Mathematical Set Operations

Python sets support standard mathematical operations.

8.1 Union

The union of two sets contains everything that appears in either set.

$$A \cup B$$

```

1 A.union(B)

```

8.2 Intersection

The intersection contains only elements that appear in both sets.

$$A \cap B$$

```
1 A.intersection(B)
```

8.3 Difference

The difference contains elements in one set but not the other.

$$A - B$$

```
1 A.difference(B)
```

8.4 Symmetric Difference

The symmetric difference contains elements that appear in exactly one of the two sets.

```
1 A.symmetric_difference(B)
```

9 Key Takeaways

- Lists are ordered, mutable collections.
- Lists can hold mixed data types, but that flexibility should be used carefully.
- Tuples are ordered but immutable.
- Named tuples make tuple-based code easier to read.
- Sets are unordered collections of unique values.
- Sets are useful for removing duplicates and performing mathematical set operations.

CPSC 250

Dictionaries

Edward Brash

1 Introduction

Dictionaries are one of the most powerful and useful data structures in Python.

Many computer scientists use dictionaries constantly because they provide an efficient way to map one piece of information to another.

Students often struggle with dictionaries initially, but they are extremely useful once understood properly.

2 When Should You Use a Dictionary?

A dictionary is useful whenever there exists a natural one-to-one mapping between two collections of information.

For example:

- names $\rightarrow$ ages
- student IDs $\rightarrow$ GPAs
- usernames $\rightarrow$ passwords
- countries $\rightarrow$ capitals

If there is a relationship between two kinds of data, a dictionary may be an appropriate structure.

3 Example Using Two Lists

Suppose we initially store information in two separate lists:

```
names = ['Bob', 'Alice', 'Jane', 'Fred']
ages = [18, 31, 16, 47]
```

The problem is that the relationship between the lists must be maintained manually.

For example:

- names[0] corresponds to ages[0]
- names[1] corresponds to ages[1]

This becomes cumbersome and error-prone.

4 Replacing with a Dictionary

Instead, we can directly store the relationship:

```
people = {  
    'Bob': 18,  
    'Alice': 31,  
    'Jane': 16,  
    'Fred': 47  
}
```

This preserves the correlation between the data naturally.

5 Accessing Dictionary Values

Dictionary values are accessed using keys:

```
print(people['Bob'])
```

Output:

```
18
```

6 Important Terminology

In a dictionary:

```
'Bob' : 18
```

- 'Bob' is called the **key**
- 18 is called the **value**

Dictionaries store **key/value pairs**.

7 Keys and Values Can Have Different Types

Keys and values do not need to have the same type.

Example:

```
friends = {  
    'Bob': 'Alice',  
    'Jane': 'Heather',  
    'Fred': 'John'  
}
```

Another example:

```
finish_order = {  
    1: 'Bob',  
    2: 'Fred',  
    3: 'Bill'  
}
```

```
print(finish_order[2])
```

Output:

```
Fred
```

Important:

The value inside square brackets is a **key**, not an index.

8 Useful Dictionary Methods

- `my_dict.keys()` — return all keys
- `my_dict.values()` — return all values
- `my_dict.items()` — return all key/value pairs
- `my_dict.get(key)` — safely retrieve a value
- `my_dict.update(other_dict)` — merge dictionaries
- `my_dict.pop(key)` — remove a key/value pair

9 Mutability

Dictionaries are mutable structures.

This means:

- values can be changed,
- new entries can be added,
- entries can be removed.

Example:

```
people['Bob'] = 19  
people['Mary'] = 27
```

10 Summary of Python Data Structures

Type	Description	Mutable?
int	Numeric integer	Yes*
float	Floating point number	Yes*
string	Sequence of characters	No
list	Ordered sequence	Yes
tuple	Ordered immutable sequence	No
set	Unordered unique collection	Yes
dict	Key/value mapping	Yes

*Python integers and floats are technically objects. Operations create new objects rather than modifying existing ones in place.

11 Key Takeaways

- Dictionaries store mappings between keys and values.
- They are ideal for representing related information.
- Keys are used for lookup instead of numeric indexing.
- Dictionaries are mutable and highly flexible.
- Dictionaries are among the most useful data structures in Python.

CPSC 250

Branching and Flow Control

Edward Brash

1 Introduction

One of the most important ideas in programming is the ability for a program to make decisions.

Programs are not simply sequences of instructions executed line-by-line forever.

Instead, programs often need to:

- make decisions,
- repeat actions,
- and alter execution flow dynamically.

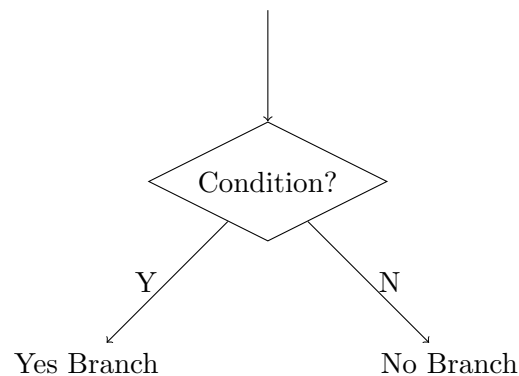
This is called:

flow control

2 Branching

Branching occurs whenever a program must decide between multiple execution paths.

Conceptually:



3 The if-else Structure

The most general branching structure in Python is:

```
if condition:
    # do something
else:
    # do something else
```

Sometimes the `else` branch simply does nothing.
In that case:

```
if condition:
    # do something
```

is really just shorthand for:

```
if condition:
    # do something
else:
    pass
```

4 Conditions Evaluate to Booleans

A condition must evaluate to a Boolean value:

`True`

or

`False`

Example:

```
a == 4
```

This expression evaluates to:

- `True` if `a` equals 4
- `False` otherwise

5 Comparison vs Assignment

A very common mistake for beginners is confusing:

```
==
```

with:

```
=
```

These are completely different operations.

5.1 Logical Comparison

```
a == 4
```

asks:

“Is the current value of **a** equal to 4?”

5.2 Assignment

```
a = 4
```

means:

“Store the value 4 into the memory location associated with **a**.”

6 Loops

Loops allow a section of code to execute repeatedly.

There are two major loop structures in Python:

1. `for` loops
2. `while` loops

7 For Loops

A `for` loop is useful when you know approximately how many times you want to execute a loop.

Example:

```
for i in range(10):  
    print(i)
```

This loop executes 10 times:

$$i = 0, 1, 2, \dots, 9$$

7.1 Looping Through Collections

```
l = ['a', 'b', 'c']  
  
for item in l:  
    print(item)
```

Output:

```
a  
b  
c
```

8 While Loops

A **while** loop is useful when you do *not* know in advance how many iterations are required.

General structure:

```
while condition:
    # do stuff
```

The loop continues executing while the condition remains true.

Example:

```
a = 0

while a < 4:
    print(a)
    a += 1
```

The loop terminates once the condition becomes false.

9 Infinite Loops

One danger with **while** loops is accidentally creating infinite loops.

Example:

```
while True:
    print("Oops")
```

This loop never terminates unless interrupted externally.

10 Break Statements

Sometimes loops contain multiple conditions for termination.

Example problem:

Loop through a list until reaching the value "end".

```
l = ["bob", "alice", "fred", "end", "jane"]

for item in l:
    if item == "end":
        break

    print(item)
```

Output:

```
bob
alice
fred
```

The **break** statement immediately exits the loop.

11 Key Takeaways

- Branching controls program decision-making.
- Conditions evaluate to Boolean values.
- `==` performs comparison.
- `=` performs assignment.
- `for` loops are ideal for known iteration counts.
- `while` loops are ideal for condition-based iteration.
- `break` immediately exits a loop.

CPSC 250

Functions

Edward Brash

1 Introduction

Functions are one of the most important organizational tools in programming.

Functions allow us to:

- divide problems into smaller pieces,
- reuse code,
- improve readability,
- and reduce duplication.

2 Basic Function Structure

General syntax:

```
def my_function(arguments):  
    # code here  
    return values
```

Important parts:

- Function name
- Argument list
- Function body
- Return value list

2.1 Argument List

The argument list contains information passed *to* the function.

2.2 Return Value List

The return value list contains information returned *from* the function.

3 Conceptual Model

Conceptually:

Main Program $\rightarrow$ Function

Arguments flow into the function.

Return values flow back out.

4 Example: add() Function

```
def add(a, b):  
    c = a + b  
    return c
```

Example usage:

```
x = 4  
y = 6  
  
z = add(x, y)  
  
print(z)
```

Output:

```
10
```

5 Very Important Idea: Pass by Value

Inside the function:

- **a** and **b** are copies of **x** and **y**
- they are different variables in memory

Changing variables inside the function does not change variables in the main program.

This is called:

pass by value

6 Naming Convention Recommendation

A very good habit is to use different variable names inside functions than in the main program.

Example:

Main Program	Function
x, y, z	a, b, c

This reinforces the idea that these are different variables occupying different memory locations.

7 Dynamic Typing in Python

Unlike languages such as:

- C
- C++
- Java
- Go

Python uses dynamic typing (sometimes informally called “duck typing”).
The same function may operate on multiple data types.

Example:

```
def add(a, b):  
    return a + b
```

This works for integers:

```
print(add(4, 6))
```

but also for strings:

```
x = "that_is"  
y = "so_cool!"  
  
z = add(x, y)  
  
print(z)
```

Output:

```
that is so cool!
```

Python automatically determines the correct behavior based on object types.

8 Variable Scope

Scope determines where variables have meaning inside a program.

8.1 Example

```
employee_name = 'Bob'  
  
def get_name():  
    name = input()  
    employee_name = name  
  
get_name()  
  
print(employee_name)
```

Important:
The variable:

```
employee_name
```

inside the function is different from the variable outside the function.
Thus, the printed value remains:

```
Bob
```

unless special mechanisms such as `global` are used.

9 Pass by Reference?

Students often hear the phrase:

“Python passes lists by reference.”

This is not quite correct.

10 The Common Misconception

Suppose we have:

```
numbers = [1, 2, 3, 4, 5]
```

and:

```
def change_numbers(nums):  
    nums[0] = 7
```

Then:

```
change_numbers(numbers)  
print(numbers)
```

produces:

```
[7, 2, 3, 4, 5]
```

Many people incorrectly conclude that Python uses pass-by-reference.

11 What is Actually Happening

The important distinction is:

- the list itself exists as a mutable object in memory,
- the function receives a copy of the reference,
- both variables point to the same underlying object.

The variables themselves are still separate variables.

12 Mutable vs Immutable Objects

This behavior depends heavily on mutability.

12.1 Immutable Objects

Examples:

- integers
- floats
- strings
- tuples

cannot be modified in-place.

12.2 Mutable Objects

Examples:

- lists
- dictionaries
- sets

can be modified in-place.

13 Key Takeaways

- Functions help organize and reuse code.
- Arguments pass information into functions.
- Return values pass information back out.
- Python primarily behaves like pass-by-value.
- Mutable objects can create reference-sharing behavior.
- Scope determines where variables exist and have meaning.
- Python uses dynamic typing.

CPSC 250

Big Program One: Constructing a 32-Bit Floating Point Representation

Edward Brash

1 Purpose of the Program

In this set of notes, we will build a larger Python program whose purpose is to illustrate how a real number can be represented in binary floating point form.

The goal is not to replace Python's built-in floating point system. Python already does this for us. Instead, the goal is pedagogical:

We want to understand, step by step, how a decimal floating point number can be decomposed into a binary representation involving a sign bit, exponent bits, and mantissa bits.

This program brings together many of the ideas reviewed earlier in the course.

2 Skills Needed

To build this program, we will need several pieces of Python knowledge.

1. Knowledge of primitive data types: integers, floats, and strings.
2. Knowledge of functions: functions we write ourselves, built-in functions, and class methods.
3. Knowledge of how floating point numbers are stored.
4. Loops: both `for` loops and `while` loops.
5. Branching: `if`, `else`, and logical conditions.
6. Getting input from the user.
7. Producing formatted output using `print()`.

3 Overall Program Design

The program will ask the user for a decimal floating point number, then construct an approximate binary floating point representation.

The high-level design is:

1. Get a floating point number from the user.
2. Ask how many binary places should be used after the binary point.

3. Write a function that converts the integer part of the number to binary.
4. Write a function that converts the fractional part of the number to binary.
5. Determine the mantissa and exponent.
6. Construct a 32-bit floating point representation.

4 Step 1: Get a Floating Point Number

We begin by asking the user for a decimal floating point value.

```
n = input("Enter your floating point value: ")
```

Important:

The value returned by `input()` is always a string.

Therefore, if the user enters 3.125, then initially Python stores the text string "3.125", not the numeric value 3.125. To treat this value numerically, we convert it:

```
n = float(input("Enter your floating point value: "))
```

5 Step 2: Ask for the Number of Binary Decimal Places

Next, we ask how many places should be included after the binary point.

For example, the decimal number

$$0.1_{10}$$

has a binary expansion that does not terminate cleanly:

$$0.1_{10} = 0.0001100110011 \dots_2$$

Therefore, in a practical program, we must decide how many binary digits after the point we want to keep.

```
p = int(input("Enter the number of binary places in the result: "))
```

The variable `p` is an integer. For example, if `p = 6`, then we will compute six binary digits after the binary point.

6 Step 3: Write a Conversion Function

We want to write a function that takes the number `n` and the number of binary places `p`, then returns the binary representation as a string.

```
def get_binary_rep(n, p):  
    ...  
    return result
```

The output will be a string because binary representations such as 11.001000 are easiest to construct as text.

7 Working Example

We will use:

$$3.125_{10}$$

and compute the binary representation to six places after the binary point.

$$3.125_{10} = 11.001000_2$$

The rest of these notes explain how to produce this result algorithmically.

8 Breaking the Problem into Pieces

To convert a decimal number to binary, we separate it into two parts:

$$3.125 = 3 + 0.125$$

Then we handle each part separately:

1. Convert the integer part to binary.
2. Convert the fractional part to binary.
3. Join the two pieces with a binary point.

9 Converting the Integer Part to Binary

For the integer part:

$$3_{10} = 11_2$$

More generally, we convert an integer to binary using repeated integer division by 2.

9.1 Example: Converting 13 to Binary

Consider:

$$13_{10}$$

We repeatedly divide by 2 and track the remainder.

Number	Integer Division by 2	Remainder
13	$13 // 2 = 6$	$13 \% 2 = 1$
6	$6 // 2 = 3$	$6 \% 2 = 0$
3	$3 // 2 = 1$	$3 \% 2 = 1$
1	$1 // 2 = 0$	$1 \% 2 = 1$

The remainders are produced from right to left:

$$13_{10} = 1101_2$$

9.2 Algorithm

The algorithm is:

1. Start with an integer `num`.
2. Set the result string to the empty string.
3. While `num > 0`:
 - (a) Compute `digit = num % 2`.
 - (b) Add this digit to the front of the result string.
 - (c) Replace `num` with `num // 2`.

9.3 Python Function

```
def convert_integer_to_binary(num):  
    # Convert a nonnegative integer to a binary string.  
  
    if num == 0:  
        return "0"  
  
    result = ""  
  
    while num > 0:  
        digit = num % 2  
        result = str(digit) + result  
        num = num // 2  
  
    return result
```

The line

```
result = str(digit) + result
```

adds the new digit to the front of the string because the remainders appear in reverse order.

10 Converting the Fractional Part to Binary

Now consider the fractional part:

$$0.125_{10}$$

Since

$$0.125 = \frac{1}{8},$$

we know:

$$0.125_{10} = 0.001_2$$

To six places:

$$0.125_{10} = 0.001000_2$$

11 Understanding Binary Fractions

Binary fractional places represent powers of two:

$$0.b_1b_2b_3b_4\dots_2$$

means:

$$b_1 \left(\frac{1}{2} \right) + b_2 \left(\frac{1}{4} \right) + b_3 \left(\frac{1}{8} \right) + b_4 \left(\frac{1}{16} \right) + \dots$$

So:

$$0.001_2 = 0 \left(\frac{1}{2} \right) + 0 \left(\frac{1}{4} \right) + 1 \left(\frac{1}{8} \right) = 0.125_{10}$$

12 Hand Algorithm for the Fractional Part

For a fractional number, compare it to:

$$\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots$$

At each step:

- If the current number is greater than or equal to the comparison value, write 1 and subtract the comparison value.
- Otherwise, write 0.

12.1 Example: 0.125

To six places:

Place	Comparison	Current Number	Output Bit
1	$1/2 = 0.5$	0.125	0
2	$1/4 = 0.25$	0.125	0
3	$1/8 = 0.125$	0.125	1
4	$1/16 = 0.0625$	0	0
5	$1/32 = 0.03125$	0	0
6	$1/64 = 0.015625$	0	0

Thus:

$$0.125_{10} = 0.001000_2$$

12.2 Example: 0.95

For a less trivial example:

$$0.95_{10}$$

- $0.95 > 0.5$, so write 1 and subtract:

$$0.95 - 0.5 = 0.45$$

- $0.45 > 0.25$, so write 1 and subtract:

$$0.45 - 0.25 = 0.20$$

- $0.20 > 0.125$, so write 1 and subtract:

$$0.20 - 0.125 = 0.075$$

- $0.075 > 0.0625$, so write 1 and subtract:

$$0.075 - 0.0625 = 0.0125$$

So the binary fraction begins:

$$0.95_{10} \approx 0.1111\dots_2$$

The expansion continues, and because many decimal fractions do not terminate in binary, this process often produces an approximation.

13 Python Function for the Fractional Part

```
def convert_fraction_to_binary(num, places):
    # Convert the fractional part of a number to binary.

    result = ""
    count = 0

    while num > 0 and count < places:
        comparison = 2 ** (-(count + 1))

        if num >= comparison:
            result = result + "1"
            num = num - comparison
        else:
            result = result + "0"

        count = count + 1

    while len(result) < places:
        result = result + "0"

    return result
```

14 Putting the Pieces Together

Now we combine the integer and fractional pieces.

```
def get_binary_rep(n, p):  
    # Return a binary string representation of n using p fractional bits.  
  
    front = int(n)  
    back = n - front  
  
    front_bin = convert_integer_to_binary(front)  
    back_bin = convert_fraction_to_binary(back, p)  
  
    result = front_bin + "." + back_bin  
  
    return result
```

For:

```
get_binary_rep(3.125, 6)
```

the result is:

```
11.001000
```

15 From Binary Representation to Floating Point Form

Once we have a binary representation such as:

$$11.001000_2$$

we want to express it in normalized binary scientific notation:

$$1.xxxxx \times 2^E$$

For example:

$$11.001000_2 = 1.1001000_2 \times 2^1$$

Here the mantissa begins after the leading 1, the exponent is 1, and the leading 1 is implicit in normalized representation.

16 Step 5: Determine Mantissa and Exponent

Suppose the binary representation has two parts:

```
front_bin = "11"  
back_bin = "001000"
```

Since the integer part is nonzero, this is a positive-exponent case.

16.1 Positive Exponent Case

If the part before the binary point is not zero, then the leading digit will always be 1.

For example:

$$10011010.11101_2$$

The decimal point must move left until only one digit remains before the point.

If:

```
front_bin = "10011010"
```

then:

$$\text{exponent} = \text{len}(\text{front_bin}) - 1 = 8 - 1 = 7$$

The mantissa is built from:

$$\text{front_bin}[1:] + \text{back_bin}$$

because the first 1 is implicit and is not stored.

```
exponent = len(front_bin) - 1  
mantissa = front_bin[1:] + back_bin
```

16.2 Negative Exponent Case

If the integer part is zero, then the number has the form:

$$0.00101101101_2$$

Here we must search the fractional part to find the first 1.

For example:

$$0.00101101101_2 = 1.01101101_2 \times 2^{-3}$$

Thus the exponent is negative.

A clean Python approach is:

```
first_one = back_bin.find("1")  
exponent = -(first_one + 1)  
mantissa = back_bin[first_one + 1:]
```

If:

```
back_bin = "00101101101"
```

then `first_one = 2`, so:

$$E = -(2 + 1) = -3$$

17 Mantissa Padding and Truncation

The mantissa field must contain exactly 23 bits.

Therefore:

- if the mantissa is too short, pad it with zeroes;
- if the mantissa is too long, truncate it.

```
if len(mantissa) < 23:
    mantissa = mantissa + (23 - len(mantissa)) * "0"

if len(mantissa) > 23:
    mantissa = mantissa[0:23]
```

18 Step 6: Convert to 32-Bit Floating Point Representation

A 32-bit single precision style floating point representation consists of:

Field	Number of Bits	Meaning
Sign	1	positive or negative
Exponent	8	biased exponent
Mantissa	23	fractional significand bits

sign	exponent	mantissa
1 bit	8 bits	23 bits

19 The Sign Bit

The sign bit is 0 for positive numbers and 1 for negative numbers.

```
if n < 0:
    sign_bit = "1"
    n = abs(n)
else:
    sign_bit = "0"
```

20 The Exponent Field

Single precision stores the exponent using a bias of 127.

If the actual exponent is E , then the stored exponent is:

$$E + 127$$

For example, if $E = 7$, then:

$$E_{\text{stored}} = 7 + 127 = 134$$

Then 134 is converted to an 8-bit binary number.

```
exp = int(exponent) + 127
exp_binary_rep = convert_integer_to_binary(exp)
```

If the exponent binary representation has fewer than 8 bits, we pad with leading zeroes:

```
if len(exp_binary_rep) < 8:
    exp_binary_rep = (8 - len(exp_binary_rep)) * "0" + exp_binary_rep
```

21 Final 32-Bit String

Finally, the 32-bit representation is assembled as:

sign bit + exponent bits + mantissa bits

```
float_rep = sign_bit + exp_binary_rep + mantissa
```

22 Complete Program

```
def convert_integer_to_binary(num):
    # Convert a nonnegative integer to a binary string.

    if num == 0:
        return "0"

    result = ""

    while num > 0:
        digit = num % 2
        result = str(digit) + result
        num = num // 2

    return result

def convert_fraction_to_binary(num, places):
    # Convert a fractional number between 0 and 1 to binary.

    result = ""
    count = 0

    while num > 0 and count < places:
        comparison = 2 ** (-(count + 1))

        if num >= comparison:
            result = result + "1"
            num = num - comparison
        else:
            result = result + "0"

        count = count + 1
```

```

while len(result) < places:
    result = result + "0"

return result

def get_binary_rep(n, p):
    # Return binary representation of n using p fractional places.

    front = int(n)
    back = n - front

    front_bin = convert_integer_to_binary(front)
    back_bin = convert_fraction_to_binary(back, p)

    return front_bin + "." + back_bin

def normalize_binary_rep(front_bin, back_bin):
    # Return exponent and mantissa bits from binary pieces.

    if front_bin != "0":
        exponent = len(front_bin) - 1
        mantissa = front_bin[1:] + back_bin
    else:
        first_one = back_bin.find("1")

        if first_one == -1:
            exponent = 0
            mantissa = "0" * 23
        else:
            exponent = -(first_one + 1)
            mantissa = back_bin[first_one + 1:]

    if len(mantissa) < 23:
        mantissa = mantissa + (23 - len(mantissa)) * "0"

    if len(mantissa) > 23:
        mantissa = mantissa[0:23]

    return exponent, mantissa

def get_32_bit_float_rep(n, p):
    # Construct a simplified 32-bit floating point representation.

    if n < 0:
        sign_bit = "1"
        n = abs(n)
    else:
        sign_bit = "0"

    front = int(n)
    back = n - front

```

```

front_bin = convert_integer_to_binary(front)
back_bin = convert_fraction_to_binary(back, p)

exponent, mantissa = normalize_binary_rep(front_bin, back_bin)

exp = exponent + 127
exp_binary_rep = convert_integer_to_binary(exp)

if len(exp_binary_rep) < 8:
    exp_binary_rep = (8 - len(exp_binary_rep)) * "0" + exp_binary_rep

if len(exp_binary_rep) > 8:
    raise ValueError("Exponent does not fit in 8 bits.")

return sign_bit + exp_binary_rep + mantissa

n = float(input("Enter your floating point value: "))
p = int(input("Enter the number of binary places in the result: "))

binary_rep = get_binary_rep(abs(n), p)
float_rep = get_32_bit_float_rep(n, p)

print("Binary representation:", binary_rep)
print("32-bit floating point representation:", float_rep)
print("Grouped:")
print(float_rep[0], float_rep[1:9], float_rep[9:])

```

23 Testing the Program

For:

$$n = 3.125$$

we know:

$$3.125_{10} = 11.001_2$$

Normalized:

$$11.001_2 = 1.1001_2 \times 2^1$$

So:

- sign bit is 0;
- exponent is 1;
- biased exponent is $1 + 127 = 128$;
- exponent bits are 10000000;
- mantissa begins 1001 and is padded with zeroes.

Thus the 32-bit representation should be:

0 10000000 1001000000000000000000

or:

01000000010010000000000000000000

24 Important Caveats

This program is educational. It deliberately avoids several complications of the full IEEE-754 standard, including:

- rounding instead of truncation,
- subnormal numbers,
- signed zero,
- infinity,
- NaN,
- overflow and underflow handling,
- exact binary behavior of Python’s built-in floats.

Nevertheless, it captures the central idea:

$$\text{floating point number} = (-1)^s \times 1.\text{mantissa} \times 2^E$$

25 Key Takeaways

- A floating point number can be separated into integer and fractional parts.
- Integer parts can be converted to binary using repeated division by 2.
- Fractional parts can be converted using repeated comparison to powers of 1/2.
- Binary floating point numbers are normalized into the form $1.xxxxx \times 2^E$.
- A 32-bit representation stores 1 sign bit, 8 exponent bits, and 23 mantissa bits.
- The exponent is stored with a bias of 127.
- Python string operations are useful for constructing binary representations manually.

CPSC 250

Big Program Two: Credit Card Debt Search Program

Edward Brash

1 Introduction

In this project, we consider a realistic programming task involving a large CSV data file.

Suppose we are given a large CSV file containing information about individuals in the United States, including:

- last names,
- states,
- credit card debt values.

Our goal is to write a Python program that can search this data efficiently and answer questions such as:

1. Find all people whose names start with a particular string.
2. Find all people with debt below some limit.
3. Find all people from a particular state.
4. Find the person with the highest debt.

Although this may initially seem like a large and intimidating task, the key idea is:

Break the large problem into many smaller problems.

Each smaller problem is manageable and usually only requires a few lines of code.

2 Questions We Must Answer

Before writing code, we should think carefully about the design of the program.

Some important questions are:

1. What does the CSV file look like?
2. What data structures should we use?
3. What algorithms are needed for searching?

These are examples of software design questions.

3 Understanding the CSV File

Suppose our CSV file is called:

```
customer_data.csv
```

and contains rows such as:

```
Smith,VA,2450.75
Johnson,CA,10234.10
Jones,NY,500.00
Brown,TX,7890.50
```

Each row contains three pieces of information:

1. Last name,
2. State,
3. Credit card debt.

Notice that:

- names are strings,
- states are strings,
- debts are floating point numbers.

4 Choosing a Data Structure

The notes propose a very simple first design idea:

Use three ordered lists.

Specifically:

```
names = []
states = []
debts = []
```

These are called **parallel lists**.

4.1 Why Parallel Lists Work

The important idea is that the entries at the same index correspond to the same person.

For example:

Index	names	states	debts
0	Smith	VA	2450.75
1	Johnson	CA	10234.10
2	Jones	NY	500.00
3	Brown	TX	7890.50

Thus:

```
names [2]
```

corresponds to:

```
states [2]
```

and:

```
debts [2]
```

because all three refer to the same customer.

5 Step 1: Read the Data from the File

The first major task is to read the CSV file and store the information in the lists.

The notes propose writing a function:

```
def read_customer_data(filename):
```

This function should:

1. open the file,
2. read every row,
3. extract the data,
4. store the data in the lists,
5. return the lists.

6 Opening a CSV File

Python provides built-in file handling tools.

A common pattern is:

```
with open(filename, "r") as file:
```

Important ideas:

- "r" means "read mode",
- file becomes a file object,
- the with statement automatically closes the file.

7 Using the CSV Module

Python includes a built-in module called `csv` for working with CSV files.

We begin by importing it:

```
import csv
```

Then we create a CSV reader:

```
reader = csv.reader(file)
```

The reader behaves like a loop over rows.
Each row is itself a list of strings.
For example, a row might look like:

```
["Smith", "VA", "2450.75"]
```

8 Reading Rows from the File

We loop through the rows:

```
for row in reader:
```

Inside the loop:

- `row[0]` is the name,
- `row[1]` is the state,
- `row[2]` is the debt.

We append these values to the appropriate lists.

9 The `append()` Method

Python lists provide the method:

```
my_list.append(value)
```

This adds a new value to the end of the list.
For example:

```
names.append("Smith")
```

adds the string "Smith" to the end of the list.

10 Converting Debt to a Float

When data is read from a CSV file, everything initially appears as a string.

For example:

```
row[2] == "2450.75"
```

But debt values should behave numerically.
Therefore we convert them:

```
float(row[2])
```

This converts the string into a floating point number.

11 Complete File Reading Function

Putting everything together:

```
import csv

def read_customer_data(filename):

    names = []
    states = []
    debts = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        for row in reader:

            names.append(row[0])
            states.append(row[1])
            debts.append(float(row[2]))

    return names, states, debts
```

12 Returning Multiple Values

Notice the line:

```
return names, states, debts
```

Python allows functions to return multiple values.

We can capture them using:

```
names, states, debts = read_customer_data("customer_data.csv")
```

This is extremely useful when working with related data structures.

13 Step 2: Get Information from the User

Next, the program asks the user for search information.

The notes suggest three kinds of input:

1. a debt limit,
2. a search phrase,
3. a state abbreviation.

For example:

```
debt_limit = float(input("Enter debt limit: "))
search_phrase = input("Enter search phrase: ")
st = input("Enter state abbreviation: ")
```

Notice:

- debt limits should be converted to numbers,
- names and states remain strings.

14 Searching for Names

Suppose we want all names beginning with:

```
"Jo"
```

We loop through the list of names.

```
for n in range(len(names)):
```

The expression:

```
range(len(names))
```

produces indices:

0, 1, 2, 3, ...

Using indices is important because we need access to the matching state and debt information.

15 The startswith() String Method

Python strings provide a method called:

```
startswith()
```

Example:

```
if names[n].startswith(search_phrase):
```

This checks whether the string begins with the desired text.
For example:

```
"Johnson".startswith("Jo")
```

returns:

```
True
```

while:

```
"Smith".startswith("Jo")
```

returns:

```
False
```

16 Complete Name Search Example

```
print("Matching names:")
for n in range(len(names)):
    if names[n].startswith(search_phrase):
        print(names[n], states[n], debts[n])
```

17 Searching by Debt

Suppose we want everyone with debt less than \$2000.

We compare each debt value against the limit.

```
print("People below debt limit:")
for n in range(len(debts)):
    if debts[n] < debt_limit:
        print(names[n], states[n], debts[n])
```

Notice that this is a simple linear search through the data.

18 Searching by State

Suppose we want all customers from Virginia.

```
print("People from state:", st)
for n in range(len(states)):
    if states[n] == st:
        print(names[n], states[n], debts[n])
```

This uses the equality operator:

```
==
```

to compare strings.

19 Step 3: Find the Highest Credit Card Debt

Another important task is finding the maximum debt value.

The notes use a classic algorithm based on tracking the index of the largest value found so far.

20 Understanding the Maximum Search Algorithm

The algorithm is:

1. Assume the first value is the maximum.
2. Loop through the remaining values.
3. If a larger value is found, update the index.
4. At the end, the stored index corresponds to the largest debt.

21 Implementing the Maximum Search

```
index_max = 0

for n in range(len(debts)):

    if debts[n] > debts[index_max]:

        index_max = n
```

Important idea:

We store the index of the maximum value, not the value itself.

This is useful because the index also gives access to the matching name and state.

22 Printing the Maximum Debt Customer

After the loop finishes:

```
print("Highest debt customer:")
print(names[index_max], states[index_max], debts[index_max])
```

Suppose:

Index	Name	State	Debt
0	Smith	VA	2450
1	Johnson	CA	10234
2	Jones	NY	500

Then:

```
index_max == 1
```

because Johnson has the largest debt.

23 Building Programs Incrementally

A very important lesson from this project is:

Large programs should be built incrementally.

Instead of trying to write everything at once:

1. Write one small piece.
2. Test it.
3. Move to the next piece.
4. Test again.

This is how real software development works.

24 Program Decomposition

At first, a 100-line program can appear overwhelming.

However:

Most large programs are really collections of many small programs.

In this project:

- reading the file is only a few lines,
- searching names is only a few lines,
- searching debts is only a few lines,
- finding a maximum is only a few lines.

The complexity comes from combining many simple ideas together.

25 Complete Example Program

```
import csv

def read_customer_data(filename):

    names = []
    states = []
    debts = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        for row in reader:

            names.append(row[0])
            states.append(row[1])
            debts.append(float(row[2]))

    return names, states, debts
```

```

# Read data from the CSV file
names, states, debts = read_customer_data("customer_data.csv")

# Get information from the user
debt_limit = float(input("Enter debt limit: "))
search_phrase = input("Enter search phrase: ")
st = input("Enter state abbreviation: ")

# Search for names
print()
print("Names beginning with", search_phrase)

for n in range(len(names)):

    if names[n].startswith(search_phrase):

        print(names[n], states[n], debts[n])

# Search by debt
print()
print("People below debt limit")

for n in range(len(debts)):

    if debts[n] < debt_limit:

        print(names[n], states[n], debts[n])

# Search by state
print()
print("People from", st)

for n in range(len(states)):

    if states[n] == st:

        print(names[n], states[n], debts[n])

# Find highest debt
index_max = 0

for n in range(len(debts)):

    if debts[n] > debts[index_max]:

        index_max = n

print()
print("Highest debt customer:")

```

```
print(names[index_max], states[index_max], debts[index_max])
```

26 Possible Improvements

This design works well for learning purposes, but there are many possible improvements.

For example:

- use dictionaries instead of parallel lists,
- use classes and objects,
- use pandas,
- sort the data,
- use binary search,
- add error checking,
- handle malformed CSV files,
- make searches case-insensitive.

However, the simple version is ideal for learning the fundamentals.

27 Computational Complexity

All searches in this program are linear searches.

That means:

$$\mathcal{O}(n)$$

time complexity.

Why?

Because in the worst case, we may need to examine every entry in the list.

For a file with:

10,000

rows, this is usually perfectly acceptable.

For millions of rows, more sophisticated data structures might become necessary.

28 Key Takeaways

- CSV files store structured tabular data.
- The `csv` module helps read CSV files safely.
- Parallel lists are a simple but useful data structure.
- File data is initially read as strings.

- Numerical values often require conversion using `float()` or `int()`.
- Linear searches are straightforward to implement.
- The `startswith()` method is useful for string searching.
- Maximum search algorithms often track indices rather than values.
- Large programs become manageable when broken into smaller pieces.

Week 2: Object-Oriented Programming

CPSC 250

Philosophy of Complex Objects

Edward Brash

1 Motivation

Before we begin writing our own classes, it is useful to think about why classes exist in the first place.

A program often needs to represent a “thing” from the real world. That thing may not be naturally described by a single integer, float, string, list, tuple, or dictionary. Instead, it may require several related pieces of information, possibly of different data types.

For example, imagine that we want to store information about a student.

2 A Student as a Collection of Information

A student might have the following data associated with them:

Number	Information	Type
1	First name	string
2	Last name	string
3	Student ID	integer
4	Address	list
5	Grades	dictionary

The address itself might be a list:

```
address = [908, "Promenade Lane", "Williamsburg", "VA", 23185]
```

The grades might be a dictionary:

```
grades = {  
    "HW": 94.2,  
    "Quiz": 75.8,  
    "Tests": 88.4,  
    "Final": 74.2  
}
```

Already, this is no longer a simple primitive value. It is a structured collection of information.

3 The List-Based Approach

One possible approach is to store all the student information in a list:

```
student_info = [first_name, last_name, student_id, address, grades]
```

Then we could access the homework grade by writing:

```
print(student_info[4]["HW"])
```

This works, but it requires us to remember that index 4 contains the grades dictionary.

4 Why This Works, But Is Not Ideal

The list-based approach technically works.

However, it has several disadvantages:

- The meaning of each index must be remembered.
- The code becomes difficult to read.
- The structure is fragile.
- If we change the order of the list, many parts of the program may break.

For example:

```
student_info[4]["HW"]
```

requires us to remember that:

- index 4 means “grades”,
- and "HW" is a key inside the grades dictionary.

This can quickly become confusing.

5 A Complex Object

The key idea is that `student_info` is really a complex assembly of information.

It contains:

- strings,
- integers,
- lists,
- dictionaries,
- and possibly even other objects.

It is natural to call this entire collection an **object**.

6 Everything in Python Is Already an Object

In Python, almost everything can be thought of as an object:

- integers,
- floats,
- strings,
- booleans,
- lists,
- tuples,
- dictionaries,
- lists of lists,
- lists of dictionaries,
- dictionaries of lists,
- and so on.

Python already treats these things as objects internally.
The next step is to ask:

Can we define our own object types?

The answer is yes.

7 Main Point

It would be useful to have a system that allows us to:

1. define objects in our own way,
2. decide what information belongs inside those objects,
3. define how users interact with those objects,
4. and protect the internal structure from accidental misuse.

This is the motivation for classes and object-oriented programming.

8 Classes and Object-Oriented Programming

A class allows us to define a new kind of object.

Instead of forcing student data into a list, we could imagine defining a **Student** class.

Then each student would be a **Student** object, with meaningful internal variables such as first name, last name, ID number, address, and grades.

The advantage is that the program becomes much more readable and maintainable.

9 Key Takeaways

- Real-world data often consists of many related pieces of information.
- Primitive types alone are not always enough.
- Lists and dictionaries can store complex information, but may become awkward.
- A complex assembly of information can be thought of as an object.
- Classes allow us to define our own object types.
- Object-oriented programming is about defining objects and controlling how users interact with them.

CPSC 250

Classes and Objects

Edward Brash

1 Object-Oriented Programming

Object-oriented programming is a style of programming based on the idea that a program can be organized around objects.

An object combines:

- data,
- functions that operate on that data,
- and rules for how outside code should interact with that data.

This gives us a powerful way to manage complexity.

2 Concept 1: Encapsulation

One of the central ideas of object-oriented programming is **encapsulation**.

Encapsulation means hiding internal variables from the user and requiring the user to interact with an object through well-defined methods.

2.1 Why Encapsulation Is Good

Encapsulation helps with:

- maintenance,
- debugging,
- team software development,
- changing internal implementation details without breaking user code.

The idea is that the user of a class should not need to know exactly how the class works internally. They should only need to know how to use its public methods.

3 Good and Bad Design

Good design hides unnecessary details.

Bad design exposes too much internal structure and forces users to depend on details that should not matter.

A good object acts like a carefully designed interface.

The user should interact with the object through functions, not by manually changing internal data.

4 Objects and Classes

4.1 Objects

An object is a grouping of variables and functions that act on those variables.

4.2 Classes

A class is a template used to create objects.

A class defines:

- what variables an object will contain,
- what functions operate on those variables,
- and how the object should behave.

In this sense, a class is the DNA of object-oriented programming.

5 Example: A Restaurant Object

Suppose we want to build a restaurant rating system, similar in spirit to Yelp.

For each restaurant, we might want to store:

Information	Example	Type
Name	Moe's	string
Rating	7	integer
Price	***	string
Cuisine type	Mexican	string

In addition, we might want functions that act on those variables:

- set the name,
- get the name,
- set the rating,
- get the rating,
- set the price,
- get the price,
- set the cuisine,
- get the cuisine,
- print a report.

6 Defining a Restaurant Class

A first version of the class might look like this:

```
class Restaurant:

    def __init__(self):
        self.name = "none"
        self.rating = -1
        self.price = "none"
        self.cuisine = "none"

    def set_name(self, user_name):
        self.name = user_name

    def get_name(self):
        return self.name

    def set_rating(self, user_rating):
        self.rating = user_rating

    def get_rating(self):
        return self.rating

    def set_price(self, user_price):
        self.price = user_price

    def get_price(self):
        return self.price

    def set_cuisine(self, user_cuisine):
        self.cuisine = user_cuisine

    def get_cuisine(self):
        return self.cuisine

    def print_info(self):
        print("Restaurant Information")
        print(f"Name: {self.name}")
        print(f"Rating: {self.rating}")
        print(f"Price: {self.price}")
        print(f"Cuisine Type: {self.cuisine}")
```

7 Using the Class

In the main program, we can create an object:

```
moes = Restaurant()
```

Then we can interact with the object using methods:

```
moes.set_name("Moe's")
moes.set_rating(7)
moes.set_price("***")
moes.set_cuisine("Mexican")
```

```
moes.print_info()
```

8 The Constructor

The method:

```
def __init__(self):
```

is called the constructor.

It is called automatically whenever a new object of the class is created.

9 The Meaning of self

Every method in a class usually has **self** as the first parameter.

The variable **self** refers to the particular object that is calling the method.

For example:

```
moes.set_name("Moe's")
```

means that inside **set_name**, **self** refers to **moes**.

Thus:

```
self.name = user_name
```

means:

Set the name belonging to this particular object.

10 Public Access vs Good Practice

In Python, instance variables are technically public unless special naming conventions are used.

That means it is possible to write:

```
print(moes.name)
```

However, this is bad practice in introductory object-oriented design.

Instead, prefer:

```
print(moes.get_name())
```

and:

```
moes.set_name("Moe's")
```

This preserves encapsulation.

11 Instance Attributes

Variables such as:

- **self.name**,

- `self.rating`,
- `self.price`,
- `self.cuisine`

are called **instance attributes**.
They belong to each individual object.

12 Class Attributes

A class attribute belongs to the class as a whole rather than to a single object.
For example:

```
class MarathonRunner:

    race_distance = 42.195

    def __init__(self):
        self.name = "none"
        self.time = 0.0
```

Here, `race_distance` applies to all marathon runners.

```
runner1 = MarathonRunner()
runner2 = MarathonRunner()

print(runner1.race_distance)
print(runner2.race_distance)
print(MarathonRunner.race_distance)
```

All three print the same class-level value.

13 Key Takeaways

- A class is a template for creating objects.
- An object contains data and methods.
- Encapsulation hides internal details.
- Constructors initialize objects.
- `self` refers to the current object.
- Instance attributes belong to individual objects.
- Class attributes belong to the class as a whole.

CPSC 250

Class Constructors

Edward Brash

1 Review

In the previous class examples, the constructor was kept as simple as possible.

For example:

```
def __init__(self):
    self.name = "none"
    self.rating = -1
    self.price = "none"
    self.cuisine = "none"
```

This constructor creates an object and assigns default values to all member variables. That works, but it is not always convenient.

2 The Problem

Often, when we create an object, we already know some or all of the information that should go inside it.

For example, rather than writing:

```
moes = Restaurant()

moes.set_name("Moe's")
moes.set_rating(3)
moes.set_price("***")
moes.set_cuisine("Mexican")
```

it would be nicer to write:

```
moes = Restaurant("Moe's", 3, "***", "Mexican")
```

This is possible by adding parameters to the constructor.

3 Constructor Parameters

We can define the constructor as:

```
class Restaurant:

    def __init__(self, name, rating, price, cuisine):
        self.name = name
        self.rating = rating
        self.price = price
        self.cuisine = cuisine
```

Now the object can be initialized directly:

```
moes = Restaurant("Moe's", 3, "***", "Mexican")
```

4 Important Notes

4.1 Parameter Names

It is common to use the same names for constructor parameters and instance attributes:

```
def __init__(self, name, rating, price, cuisine):
    self.name = name
    self.rating = rating
    self.price = price
    self.cuisine = cuisine
```

This is not a conflict.

The name on the right side is the parameter. The name on the left side after `self.` is the instance attribute.

4.2 Documentation

It is good practice to include documentation explaining the parameters.

```
class Restaurant:
    # Represents a restaurant.
    #
    # Parameters:
    #     name: Name of the restaurant.
    #     rating: Rating from 1 to 10.
    #     price: Price category such as "$", "$$", or "$$$".
    #     cuisine: Type of cuisine.
```

This documentation can be viewed using:

```
help(Restaurant)
```

5 Default Constructor Values

Sometimes we want to provide default values for some constructor arguments.

Example:

```
class Restaurant:

    def __init__(self, name, cuisine, rating=-1, price="none"):
        self.name = name
        self.cuisine = cuisine
        self.rating = rating
        self.price = price
```

Now we can write:

```
moes = Restaurant("Moe's", "Mexican")
```

and the object will use default values for `rating` and `price`.

6 Important Rule About Defaults

Non-default parameters must come before default parameters.

This is valid:

```
def __init__(self, name, cuisine, rating=-1, price="none"):
```

This is not valid:

```
def __init__(self, name="none", cuisine, rating=-1):
```

Python needs all required parameters first, followed by optional parameters.

7 Overriding Default Values

Default values are convenient, but they can still be overridden.

```
panera = Restaurant("Panera", "Soups and Sandwiches", 4, "$$$")
five_guys = Restaurant("Five Guys", "Burgers", 10, "$$$$$")
```

These calls supply explicit values for all fields.

8 Best Practice: Give Defaults to All Member Variables

A common design choice is to make sure every member variable has a reasonable value after construction.

This prevents partially initialized objects.

For example:

```
class Restaurant:

    def __init__(self, name="none", cuisine="none",
                  rating=-1, price="none"):
        self.name = name
        self.cuisine = cuisine
        self.rating = rating
        self.price = price
```

Then all of the following make sense:

```
Restaurant()
Restaurant("Moe's")
Restaurant("Moe's", "Fake Mex")
Restaurant("Moe's", "Fake Mex", 6)
Restaurant("Moe's", "Fake Mex", 6, "$$")
```

9 Internal Helper Methods

Another best practice is to use an underscore to indicate methods that are intended to be internal.

For example, suppose we define a rectangle:

```
class Rectangle:

    def __init__(self, length=1, width=1):
```

```
        self.length = length
        self.width = width

    def _get_area(self):
        return self.length * self.width

    def resize(self, new_length):
        area = self._get_area()
        self.length = new_length
        self.width = area / new_length

    def print_rectangle(self):
        print(f"Length: {self.length:.3f}")
        print(f"Width: {self.width:.3f}")
        print(f"Area: {self._get_area():.3f}")
```

The method `_get_area()` is still technically accessible from outside the class, but the underscore indicates:

This method is intended for internal use.

10 A Note About Naming Conventions

When we see a method beginning with an underscore, such as:

```
_get_area()
```

we should not assume it is magical or special.

It is mostly a convention.

Python programmers use this convention to communicate design intent.

11 Key Takeaways

- Constructors initialize objects.
- Constructors can take parameters.
- Parameter values can be assigned to instance attributes.
- Default parameter values make object creation more flexible.
- Required parameters must come before default parameters.
- Documentation helps users understand constructor parameters.
- Leading underscores indicate internal-use methods by convention.

CPSC 250

Class Customization

Edward Brash

1 Introduction

One of the most common things we want to do with an object is print information about it.

For example, suppose we have:

```
moes = Restaurant("Moe's", "Mexican")
```

It would be convenient if we could simply write:

```
print(moes)
```

and automatically obtain useful information about the object.

Python provides a special mechanism for doing exactly this.

2 The `__str__()` Method

Python classes can define a special method:

```
def __str__(self):
```

This method tells Python how the object should appear when printed.

The method should return a string.

Example:

```
def __str__(self):  
    return f"{self.name} ({self.cuisine})"
```

3 Automatic Behavior

Suppose we define:

```
class Restaurant:  
  
    def __init__(self, name, cuisine):  
        self.name = name  
        self.cuisine = cuisine  
  
    def __str__(self):  
        return f"{self.name} ({self.cuisine})"
```

Then:

```
moes = Restaurant("Moe's", "Mexican")  
  
print(moes)
```

automatically calls:

```
moes.__str__()
```

behind the scenes.

4 Important Note

The `__str__` method must return a single string.

It does not directly print the information itself.

Bad design:

```
def __str__(self):  
    print(self.name)
```

Correct design:

```
def __str__(self):  
    return self.name
```

The returned string is then sent to the `print()` statement.

5 Benefits of `__str__()`

The `__str__` method makes objects easier to work with.

Advantages include:

- cleaner debugging,
- easier printing,
- more readable output,
- more professional object design.

6 Example

Suppose we define:

```
class Restaurant:  
  
    def __init__(self, name, cuisine):  
        self.name = name  
        self.cuisine = cuisine  
  
    def __str__(self):  
        return f"Restaurant: {self.name}, Cuisine: {self.cuisine}"
```

Then:

```
moes = Restaurant("Moe's", "Mexican")  
  
print(moes)
```

produces:

```
Restaurant: Moe's, Cuisine: Mexican
```

7 Customizing Objects

This is our first example of **class customization**.

Python allows us to redefine how many built-in operations behave for our own classes.

This leads naturally to operator overloading.

8 Transition to Operator Overloading

A natural question is:

What does it mean to add two objects together?

For example:

```
moes = Restaurant()  
panera = Restaurant()  
  
fusion = moes + panera
```

What should that mean?

The answer depends on the class.

For our own classes, we can define the rules ourselves.

9 Algebra and Operators

Algebra is really a set of rules for manipulating symbols and objects.

Operators include:

- arithmetic operators:

$+, -, *, /, \%$

- comparison operators:

$<, >, <=, >=, ==, !=$

Python allows us to redefine these operators for our own classes.

10 Example: A Time Class

Suppose we define a Time class:

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes

    def __str__(self):
        return f"{self.hours}:{self.minutes}"
```

Now we can print time objects naturally.

```
t1 = Time(10, 40)

print(t1)
```

11 Comparison Operators

Suppose we want to compare two time objects.

For example:

```
time1 < time2
```

Python does not automatically know how to compare two custom objects.
We must define the comparison ourselves.

12 The __lt__() Method

The special method:

```
def __lt__(self, other):
```

defines the meaning of the less-than operator:

<

for objects of the class.

13 Example Implementation

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes

    def __str__(self):
        return f"{self.hours}:{self.minutes}"

    def __lt__(self, other):
```

```

    if self.hours < other.hours:
        return True

    elif self.hours == other.hours:

        if self.minutes < other.minutes:
            return True

    return False

```

14 Using the Comparison

Now suppose:

```

time1 = Time(10, 40)
time2 = Time(12, 15)
time3 = Time(9, 15)

```

We can search for the earliest time:

```

min_time = time1

if time2 < min_time:
    min_time = time2

if time3 < min_time:
    min_time = time3

print(f"Earliest time is {min_time}")

```

Because we defined `__lt__`, Python now knows how to compare `Time` objects.

15 Rich Comparison Operators

Python provides several comparison customization methods.

Method	Operator
<code>__lt__</code>	<code><</code>
<code>__le__</code>	<code><=</code>
<code>__gt__</code>	<code>></code>
<code>__ge__</code>	<code>>=</code>
<code>__eq__</code>	<code>==</code>
<code>__ne__</code>	<code>!=</code>

16 Important Warning

Some comparison methods have default behavior, but that behavior may not match what we actually want.

Therefore:

Be careful and define comparison behavior explicitly whenever meaningful comparisons are needed.

17 Key Takeaways

- The `__str__` method controls how objects print.
- `print(object)` automatically calls `object.__str__()`.
- Operator overloading allows us to redefine operators for custom classes.
- Comparison operators are implemented using special methods such as `__lt__`.
- Different classes may define operators in different ways.

CPSC 250

Operator Overloading

Edward Brash

1 Introduction

Earlier, we discussed overloading comparison operators such as:

`<, >, ==, !=`

using methods such as:

```
__lt__()
```

However, Python also allows us to overload arithmetic operators.

2 Why Arithmetic Overloading Matters

Suppose we define a `Time` class.

We might naturally want to compute:

```
time4 = time3 - time1
```

But what does subtraction mean for time objects?

Python does not know automatically.

Therefore, we must define the meaning ourselves.

3 The `__sub__()` Method

The subtraction operator:

`-`

is controlled by:

```
def __sub__(self, other):
```

Here:

- `self` is the left object,
- `other` is the right object.

4 Example: Time Difference

Suppose we define:

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes
```

We can implement subtraction:

```
def __sub__(self, other):

    time_diff = Time(0, 0)

    if self.minutes >= other.minutes:

        time_diff.minutes = self.minutes - other.minutes

        time_diff.hours = self.hours - other.hours

    else:

        time_diff.minutes = (
            self.minutes - other.minutes + 60
        )

        time_diff.hours = (
            self.hours - other.hours - 1
        )

    return time_diff
```

5 Understanding the Logic

Suppose:

```
time1 = Time(10, 45)
time2 = Time(8, 20)
```

Then:

```
time1 - time2
```

should represent:

$$10 : 45 - 8 : 20 = 2 : 25$$

If the minutes require borrowing, we borrow one hour and add 60 minutes.

6 Returning a New Object

Notice that the subtraction method creates a new `Time` object:

```
time_diff = Time(0, 0)
```

and returns it.

This is a common pattern in object-oriented programming.

7 Other Numerical Operator Methods

Python provides many special methods for arithmetic operations.

Method	Operator
<code>--add--</code>	<code>+</code>
<code>--sub--</code>	<code>-</code>
<code>--mul--</code>	<code>*</code>
<code>--truediv--</code>	<code>/</code>
<code>--floordiv--</code>	<code>//</code>
<code>--mod--</code>	<code>%</code>
<code>--pow--</code>	<code>**</code>

8 Logical Operator Methods

There are also methods associated with logical operations.

Method	Meaning
<code>--and--</code>	logical and
<code>--or--</code>	logical or

9 Conversion Methods

Python also allows customization of built-in conversion functions.

Method	Function
<code>--abs--</code>	<code>abs()</code>
<code>--int--</code>	<code>int()</code>
<code>--float--</code>	<code>float()</code>

10 Example: Integer Conversion

Suppose we define:

```
class Time:
    def __int__(self):
        return 60*self.hours + self.minutes
```

Then:

```
t = Time(2, 30)
print(int(t))
```

would produce:

150

because 2 hours and 30 minutes equals 150 total minutes.

11 The Big Idea

Operator overloading allows objects to behave naturally.

For example:

- time subtraction behaves like subtraction,
- object comparison behaves like comparison,
- printing behaves naturally,
- conversions behave naturally.

This makes programs easier to read and easier to use.

12 Key Takeaways

- Python allows arithmetic operator overloading.
- Special methods define the meaning of operators for custom classes.
- `__sub__` defines subtraction.
- Many arithmetic operators can be overloaded.
- Built-in conversion functions can also be customized.
- Overloading helps objects behave naturally and intuitively.

CPSC 250

Memory Allocation and Garbage Collection

Edward Brash

1 Introduction

Memory management is one of the most important concepts in computer science.

Many programming languages require the programmer to manually allocate and free memory.

Python attempts to automate much of this process.

However, understanding what is happening internally is still extremely valuable.

2 Two Philosophies

There are two common approaches to memory management.

2.1 Approach 1

Trust Python completely.

In this approach, we simply allow Python to:

- allocate memory,
- track objects,
- deallocate unused objects.

2.2 Approach 2

Understand the system at a deeper level.

Even though Python automates memory management, understanding the underlying model helps us:

- write better code,
- debug difficult problems,
- understand performance,
- understand why memory leaks occur.

3 The Stack and the Heap

Programs typically use two major memory regions.

3.1 The Stack

The stack is:

- highly ordered,
- fast,
- used for local variables and function calls.

The stack behaves somewhat like a stack of plates.

3.2 The Heap

The heap is:

- less ordered,
- more flexible,
- used for dynamically allocated objects.

The heap behaves more like a pile of objects spread around memory.

4 CPython and the Virtual Machine

Python programs are executed by the Python virtual machine.

The most common implementation is:

CPython

CPython handles much of the memory management automatically.

Languages using similar models include:

- Python,
- Java,
- C#,
- Perl,
- Ruby,
- Lua.

5 All Python Objects Live on the Heap

In CPython, essentially all Python objects are stored on the heap.

This includes:

- integers,
- floats,
- strings,
- lists,
- dictionaries,
- user-defined objects.

6 Reference Counting

CPython keeps track of how many variables refer to each object.

This is called the **reference count**.

When an object is created:

- memory is allocated on the heap,
- the object's reference count is initialized.

7 Garbage Collection

When the reference count of an object falls to zero:

- the object is no longer accessible,
- its memory can be reclaimed.

This process is called garbage collection.

8 Example Class

```
class Test:

    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"Deleting {self.name}")
```

The method:

```
__del__()
```

is called when the object is destroyed.

9 Example Program

```
if __name__ == "__main__":  
  
    print("Discarded object creation")  
    Test("hello1")  
  
    print("Saved object creation")  
    my_list = Test("hello2")  
  
    print("Program done")
```

10 What Happens?

10.1 Case 1

```
Test("hello1")
```

creates an object on the heap, but no variable stores a reference to it.
Therefore:

- reference count quickly falls to zero,
- the object becomes garbage collectable almost immediately.

10.2 Case 2

```
my_list = Test("hello2")
```

stores a reference to the object.
Therefore the object remains allocated until the reference disappears.

11 Reference Counting Example with Strings

Consider:

```
s1 = "Python"  
s2 = "Computer Science"  
s3 = s2  
  
s1 = "Zybooks"  
s2 = s1
```

12 Step-by-Step Interpretation

Initially:

```
s1 = "Python"
```

creates a string object on the heap with reference count 1.
Then:

```
s2 = "Computer Science"
```

creates another string object.

Next:

```
s3 = s2
```

causes both `s2` and `s3` to refer to the same object.

The reference count for "Computer Science" increases to 2.

13 Reassignment

Now consider:

```
s1 = "Zybooks"
```

The old string "Python" loses its reference.

If no references remain, its reference count becomes zero and it may be garbage collected.

14 The Garbage Collector Module

Python includes a garbage collection module:

```
import gc
```

We can inspect tracked objects:

```
gc_objects = gc.get_objects()
```

```
print(len(gc_objects))
```

This can help analyze memory behavior.

15 Circular References

Reference counting works well most of the time.

However, circular references can cause problems.

16 Dog and Cat Example

Suppose we define:

```
class Dog:

    def __init__(self, name, cousin=None):
        self.name = name
        self.cousin = cousin

class Cat:

    def __init__(self, name, cousin=None):
        self.name = name
        self.cousin = cousin
```

Now:

```
bailey = Dog("Bailey")
misty = Cat("Misty", bailey)

bailey.cousin = misty
```

creates a circular reference.

17 The Problem

Now:

- Bailey refers to Misty,
- Misty refers to Bailey.

Even if we delete one variable:

```
del bailey
```

the objects may still refer to each other internally.
Their reference counts may never fall to zero.

18 Memory Leaks

This situation can create a memory leak.

A memory leak occurs when memory remains allocated even though the programmer no longer intends to use it.

19 Breaking the Circular Reference

One solution is to manually remove the reference:

```
misty.cousin = None
```

Now the reference cycle is broken.

20 Why This Matters

Programs that repeatedly leak memory may eventually:

- slow down,
- consume large amounts of RAM,
- crash.

21 Key Takeaways

- Python automatically manages most memory allocation.
- CPython uses reference counting.
- Objects live on the heap.
- The stack and heap serve different purposes.
- Objects with zero references may be garbage collected.
- Circular references can cause memory leaks.
- The `gc` module can help inspect memory behavior.

CPSC 250

Classes Program Example: The GVDie Craps Game

Edward Brash

1 Program Goal

In this example, we write a program to play an automated dice game using a provided `GVDie` class. The player rolls two dice. Depending on the result, the player may:

- win one credit,
- lose one credit,
- or set a goal for future rolls.

A round ends when the player either wins or loses a credit. The entire game ends when the player reaches zero credits.

2 The Provided GVDie Class

The provided die class represents a single six-sided die.

A simplified version is:

```
import random

class GVDie:

    def __init__(self):
        self.value = None

    def roll(self):
        self.value = random.randint(1, 6)

    def get_value(self):
        return self.value

    def compare_to(self, d):
        return self.value == d.value
```

3 The Meaning of None

The constructor initializes:

```
self.value = None
```

The value `None` means that the die does not yet have a meaningful value. This is useful because a die should not really have a value until it has been rolled.

4 Importing and Using the Die Class

If the class is stored in a file called `gv_die.py`, then the main program can import it:

```
import random
from gv_die import GVDie
```

Then we can create die objects:

```
die1 = GVDie()
die2 = GVDie()
```

Each die object has its own internal value.

5 Random Seeds

The program may ask the user for a random seed:

```
seed = int(input("Enter seed: "))
random.seed(seed)
```

A seed defines the particular sequence of random numbers.

This is useful for testing because it makes program behavior repeatable.

6 Credits

The user also enters the number of credits:

```
credits = int(input("Enter starting credits: "))
```

These credits can be interpreted as the player's number of lives.

The game continues until:

```
credits == 0
```

7 Basic Game Setup

The notes outline the beginning of the program as follows:

1. Create two `GVDie` objects.
2. Initialize the number of rounds.
3. Initialize the goal value to `-1`.
4. Begin the main game loop.

```
die1 = GVDie()
die2 = GVDie()

rounds = 0
goal = -1

while credits > 0:
    ...
```

8 How a Round Works

A round begins with an initial roll of both dice.

```
die1.roll()
die2.roll()

roll = die1.get_value() + die2.get_value()
```

The total determines what happens next.

9 Initial Roll Outcomes

For the initial roll:

- rolling 7 or 11 wins one credit,
- rolling 3 or 12 loses one credit,
- any other value becomes the goal.

```
if roll == 7 or roll == 11:
    credits = credits + 1

elif roll == 3 or roll == 12:
    credits = credits - 1

else:
    goal = roll
```

10 Goal Rolls

If the initial roll sets a goal, the player continues rolling.

On later rolls:

- rolling 7 loses one credit,
- rolling the goal wins one credit,
- any other roll means roll again.

```
while goal != -1:

    die1.roll()
    die2.roll()

    roll = die1.get_value() + die2.get_value()

    if roll == 7:
        credits = credits - 1
        goal = -1

    elif roll == goal:
        credits = credits + 1
        goal = -1
```

11 Why This Is a Class Example

This program is useful because it shows objects in action.

Instead of treating dice as raw integers, we use die objects that know how to:

- roll themselves,
- store their current value,
- return their value,
- compare themselves to another die.

The class bundles together both data and behavior.

12 Complete Program Skeleton

```
import random
from gv_die import GVDie

seed = int(input("Enter seed: "))
random.seed(seed)

credits = int(input("Enter starting credits: "))

die1 = GVDie()
die2 = GVDie()

rounds = 0

while credits > 0:

    rounds = rounds + 1
    goal = -1

    die1.roll()
    die2.roll()

    roll = die1.get_value() + die2.get_value()

    if roll == 7 or roll == 11:
        credits = credits + 1

    elif roll == 3 or roll == 12:
        credits = credits - 1

    else:
        goal = roll

    while goal != -1:

        die1.roll()
        die2.roll()
```

```
roll = die1.get_value() + die2.get_value()

if roll == 7:
    credits = credits - 1
    goal = -1

elif roll == goal:
    credits = credits + 1
    goal = -1

print("Game over")
print("Rounds played:", rounds)
```

13 Key Takeaways

- Objects combine state and behavior.
- A die object stores its current value.
- Methods such as `roll()` and `get_value()` define how users interact with the object.
- A random seed makes simulations repeatable.
- Larger programs become manageable when broken into loops, branches, and objects.

CPSC 250

Airline Seat Reservation System

Edward Brash

1 Program Goal

In this example, we design a simple airline seat reservation system.

The goal is to use classes and objects to model seats on an airplane.

Suppose the airplane has five seats. We will create a list containing five `Seat` objects.

2 Step 1: Identify Internal Variables

For each seat reservation, we want to store:

1. first name,
2. last name,
3. amount paid.

These become the internal variables of the class.

3 Step 2: Identify Class Methods

The `Seat` class should provide methods that allow us to:

1. reserve a seat,
2. unreserve a seat,
3. check whether the seat is empty,
4. print seat information.

The user should not directly manipulate the internal variables.

Instead, the user should interact with the object through methods.

4 The Seat Class

A reasonable `Seat` class is:

```

class Seat:

    def __init__(self):
        self.first_name = ""
        self.last_name = ""
        self.amount_paid = 0.0

    def reserve(self, first_name, last_name, amount_paid):
        self.first_name = first_name
        self.last_name = last_name
        self.amount_paid = amount_paid

    def unreserve(self):
        self.first_name = ""
        self.last_name = ""
        self.amount_paid = 0.0

    def is_empty(self):
        return self.first_name == "" and self.last_name == ""

    def print_seat(self):
        print(self.first_name, self.last_name, self.amount_paid)

```

5 Creating Seat Objects

Suppose:

```

num_seats = 5
available_seats = []

```

We can create five seat objects using a loop:

```

for i in range(num_seats):
    available_seats.append(Seat())

```

Now `available_seats` is a list of objects:

```

[Seat(), Seat(), Seat(), Seat(), Seat()]

```

Initially, all seats are empty.

6 Lists of Objects

This is an important idea:

A list can contain objects, not just primitive values.

Each element of the list is a separate `Seat` object.

For example:

```

available_seats[0]

```

is the first seat object.

```

available_seats[0].is_empty()

```

calls the `is_empty()` method for the first seat.

7 Printing Seat Status

We can write a function to print the current status of all seats.

```
def print_seats(seats):  
    seat_status = []  
  
    for i in range(len(seats)):  
        seat_status.append(seats[i].is_empty())  
  
    print("Current seat status")  
    print(seat_status)
```

Initially this might print:

```
[True, True, True, True, True]
```

because all seats are empty.

8 Printing the Passenger List

We can also print only the seats that are occupied.

```
def print_passenger_list(seats):  
    for i in range(len(seats)):  
        if not seats[i].is_empty():  
            print(f"{i}: ", end="")  
            seats[i].print_seat()
```

This demonstrates method calls on objects stored in a list.

9 Menu Driver

A menu driver repeatedly asks the user what task to perform.

The options are:

Command	Meaning
p	print the seat status
l	print the passenger list
r	reserve a new seat
q	quit

10 Menu Loop

```
command = input("Enter command: ")  
  
while command != "q":
```

```

if command == "p":
    print_seats(available_seats)

elif command == "l":
    print_passenger_list(available_seats)

elif command == "r":
    reserve_seat(available_seats)

else:
    print("Invalid command")

command = input("Enter command: ")

```

11 Reserving a Seat

The `reserve_seat()` function asks for a seat number and passenger information.

```

def reserve_seat(seats):

    seat_num = int(input("Enter seat number: "))

    if not seats[seat_num].is_empty():

        print("Seat not empty")

    else:

        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        paid = float(input("Enter amount paid: "))

        seats[seat_num].reserve(first_name, last_name, paid)

```

12 Complete Main Program

```

num_seats = 5
available_seats = []

for i in range(num_seats):
    available_seats.append(Seat())

command = input("Enter command: ")

while command != "q":

    if command == "p":
        print_seats(available_seats)

    elif command == "l":
        print_passenger_list(available_seats)

```

```
elif command == "r":
    reserve_seat(available_seats)

else:
    print("Invalid command")

command = input("Enter command: ")
```

13 Key Takeaways

- Classes are useful for modeling real-world objects.
- A seat object stores passenger information.
- Lists can store objects.
- Functions can receive lists of objects as arguments.
- Menu-driven programs use loops and branching.
- Encapsulation keeps object data organized and controlled.

CPSC 250

Craps Game Odds

Edward Brash

1 Motivation

The craps-style dice game used in the class example has an interesting property:

The probability of winning one credit is exactly equal to the probability of losing one credit.

That is:

$$P(+1) = P(-1) = \frac{1}{2}$$

This means the game behaves like a random walk.

2 Game Rules

On the initial roll:

- rolling 7 or 11 wins one credit;
- rolling 3 or 12 loses one credit;
- any other roll sets a goal and requires more rolling.

On later rolls:

- rolling 7 loses one credit;
- rolling the goal wins one credit;
- anything else means roll again.

3 Initial Roll Probabilities

The number of possible outcomes when rolling two dice is:

$$36$$

The probabilities of common sums are:

$$P(7) = \frac{6}{36} = \frac{1}{6}$$

$$P(11) = \frac{2}{36} = \frac{1}{18}$$

$$P(3) = \frac{2}{36} = \frac{1}{18}$$

$$P(12) = \frac{1}{36}$$

Therefore:

$$P(7 \text{ or } 11) = \frac{1}{6} + \frac{1}{18} = \frac{4}{18} = \frac{2}{9}$$

and:

$$P(3 \text{ or } 12) = \frac{1}{18} + \frac{1}{36} = \frac{3}{36} = \frac{1}{12}$$

4 Re-Roll Cases

If the initial roll is neither an immediate win nor an immediate loss, the game enters a re-roll state.

For the simplified version in the notes, this leads to a repeated process:

- some rolls end the round,
- some rolls continue the round.

The repeated continuation creates a geometric series.

5 Geometric Series Review

A geometric series has the form:

$$a + ar + ar^2 + ar^3 + \dots$$

If $|r| < 1$, then:

$$a + ar + ar^2 + ar^3 + \dots = \frac{a}{1 - r}$$

Equivalently:

$$\sum_{n=0}^{\infty} ar^n = \frac{a}{1 - r}$$

6 Probability of Eventually Winning

The notes derive the probability of winning as a sum involving immediate wins plus delayed wins.

The structure is:

$$P(+1) = \text{immediate win} + \text{delayed win}$$

The delayed portion includes the possibility of rolling again and again before eventually ending the round.

This repeated structure is exactly why a geometric series appears.

7 Final Result

After simplifying the geometric series, the result is:

$$P(+1) = \frac{1}{2}$$

Similarly:

$$P(-1) = \frac{1}{2}$$

Since every completed round either wins or loses one credit,

$$P(+1) + P(-1) = 1$$

and therefore, once one probability is known to be $1/2$, the other must also be $1/2$.

8 Interpretation

The game is fair in the sense that:

$$P(+1) = P(-1)$$

But this does not mean the game lasts forever.

The player's credits move up and down like a random walk until eventually reaching zero.

9 Random Walk Model

If the player begins with some number of credits, then after each round:

$$\text{credits} \rightarrow \text{credits} + 1$$

with probability $1/2$, or:

$$\text{credits} \rightarrow \text{credits} - 1$$

with probability $1/2$.

The game ends when:

$$\text{credits} = 0$$

10 Simulation Philosophy

The notes then transition from probability to simulation.

Instead of only solving analytically, we can simulate the game many times.

The idea is:

1. Run the game to completion many times.
2. Record the number of rounds in each game.
3. Record credits as a function of round number.
4. Use those results for analysis and plotting.

11 Data Structures for Simulation

For many simulations, we might use lists such as:

```
credit_result = []
round_result = []
result_number = []
```

For each simulated game, we append:

- the list of credit values,
- the list of round numbers,
- the final number of rounds.

12 Simulation Loop

A simplified structure is:

```
num_sims = 1000

credit_result = []
round_result = []
result_number = []

for i in range(num_sims):

    credit_round = []
    result_round = []

    # run one complete game here

    round_result.append(result_round)
    credit_result.append(credit_round)
    result_number.append(rounds)
```

13 Why This Matters

This example connects several important ideas:

- probability,
- loops,
- lists of lists,
- simulations,
- plotting,
- random walks.

It also gives a real reason to store data during a simulation.

14 Key Takeaways

- The dice game has equal probabilities of winning or losing one credit.
- Repeated re-rolls naturally lead to geometric series.
- The game behaves like a random walk.
- Simulations can be used to study behavior that is difficult to understand from a single run.
- Lists are useful for storing simulation histories.

CPSC 250

Plotting the Credits Profile

Edward Brash

1 Goal

After simulating many games, we want to visualize the results.

In particular, we want to plot the number of credits as a function of round number.

This is a data manipulation task:

Store the data, then plot the data.

2 What Data Do We Need?

To plot credits versus round number, we need two lists:

- x-values: round numbers,
- y-values: credits.

For example:

```
rounds = [0, 1, 2, 3, 4]
credits = [10, 11, 12, 11, 12]
```

These lists must correspond element-by-element.

3 Storing Simulation Results

For one simulation, we might store:

```
result_round = []
credit_round = []
```

At each round:

```
result_round.append(rounds)
credit_round.append(credits)
```

For many simulations, we need lists of lists:

```
round_result = []
credit_result = []
result_number = []
```

Each simulation contributes one list of round numbers and one list of credit values.

4 Matplotlib

Python's most common plotting library is Matplotlib.

The usual import is:

```
import matplotlib.pyplot as plt
```

The `pyplot` part is one submodule of Matplotlib.

By convention, we import it as:

```
plt
```

5 Basic Plot

A basic line plot is:

```
plt.plot(result_round, credit_round)
plt.show()
```

Here:

- `result_round` provides the x-values,
- `credit_round` provides the y-values.

6 Rules of Plotting

A good plot should include:

1. a descriptive title,
2. a labeled x-axis,
3. a labeled y-axis.

For this problem, a good title is:

```
plt.title("Credits vs. Round Number")
```

The axis labels are:

```
plt.xlabel("Round Number")
plt.ylabel("Credits")
```

7 Complete Single-Game Plot

```
plt.plot(result_round, credit_round)

plt.title("Credits vs. Round Number")
plt.xlabel("Round Number")
plt.ylabel("Credits")

plt.show()
```

8 Plotting Multiple Simulations

If we run many simulations, we can plot several credit histories at once.

Suppose:

```
round_result[i]
credit_result[i]
```

store the data for simulation i .

Then:

```
for i in range(num_sims):
    plt.plot(round_result[i], credit_result[i])
```

plots all simulations on the same axes.

9 Interpreting the Plot

Each line represents one simulated game.

A game begins with some initial number of credits.

Then credits move up and down until the game ends at zero credits.

This creates a random-walk-like path.

10 Histogram of Game Lengths

Another useful plot is a histogram of the number of rounds required before the game ends.

If:

```
result_number = [18422, 503, 971, ...]
```

stores the number of rounds for each simulation, then:

```
plt.hist(result_number)
plt.show()
```

creates a histogram.

11 Number of Bins

A histogram groups data into bins.

The number of bins affects the visual interpretation.

For example:

```
plt.hist(result_number, bins=20)
```

uses 20 bins.

Too few bins may hide structure. Too many bins may make the plot noisy.

12 Histogram Labels

As always, we label the plot.

```
plt.title("Distribution of Game Lengths")
plt.xlabel("Number of Rounds")
plt.ylabel("Frequency")
```

13 Complete Histogram Example

```
plt.hist(result_number, bins=20)

plt.title("Distribution of Game Lengths")
plt.xlabel("Number of Rounds")
plt.ylabel("Frequency")

plt.show()
```

14 Philosophy of the Simulation

The notes emphasize that we are not just playing a game.

We are modeling a random walk process.

This connects programming to probability, statistics, and simulation.

The general scientific workflow is:

1. define a model,
2. run the model many times,
3. store the results,
4. visualize the results,
5. interpret the behavior.

15 Key Takeaways

- Plotting requires stored x-values and y-values.
- For the credits simulation, x is round number and y is credits.
- Matplotlib is commonly imported as `plt`.
- Good plots need titles and axis labels.
- Multiple simulations can be plotted on the same axes.
- Histograms show distributions.
- The number of bins controls histogram resolution.

Week 3: Inheritance, Recursion, Algorithms, and Testing

CPSC 250

Derived Classes and Default Values

Edward Brash

1 Motivation

Object-oriented programming becomes much more powerful when we can create new classes from existing classes.

This is called **inheritance**.

A derived class inherits variables and methods from a base class, then adds more specific behavior.

2 Base Class: Transportation

Suppose we begin with a general class representing a transportation mode.

```
class Transportation:

    def __init__(self, name, speed=75):
        self.name = name
        self.speed = speed

    def info(self):
        print(f"{self.name} can go {self.speed} mph")
```

The default speed is:

75 mph

This means that if the user does not specify a speed, Python uses 75 automatically.

3 Why Use a Base Class?

There are many modes of transportation:

- motorized vehicles,
- self-powered vehicles,
- flight vehicles,
- water vehicles,
- and many others.

All of these are transportation modes, so it is useful to put common features in a shared base class.

4 Derived Class: MotorizedVehicle

A motorized vehicle is a transportation mode, but it has additional properties.

For example, it has:

- miles per gallon,
- fuel level,
- ability to add fuel,
- ability to drive.

```
class MotorizedVehicle(Transportation):

    def __init__(self, name, speed=80, mpg=40):
        Transportation.__init__(self, name, speed)

        self.mpg = mpg
        self.fuel_gal = 0

    def add_fuel(self, amount):
        self.fuel_gal += amount

    def drive(self, distance):
        required_fuel = distance / self.mpg

        if self.fuel_gal < required_fuel:
            print("Not enough fuel")

        else:
            self.fuel_gal -= required_fuel
            print(f"{self.fuel_gal} gallons remain")
```

5 Explicitly Calling the Base Constructor

The line:

```
Transportation.__init__(self, name, speed)
```

explicitly calls the constructor of the base class.

This initializes the part of the object inherited from `Transportation`.

This is extremely important: the derived class constructor does not automatically run the base class constructor unless we ask it to.

6 Derived Class: Motorcycle

A motorcycle is a specific kind of motorized vehicle.

```
class Motorcycle(MotorizedVehicle):

    def __init__(self, name, speed=55, mpg=25):
        MotorizedVehicle.__init__(self, name, speed, mpg)
```

```
def wheelie(self):  
    print("That's dangerous!")
```

The `Motorcycle` class inherits:

- `name`,
- `speed`,
- `mpg`,
- `fuel_gal`,
- `info()`,
- `add_fuel()`,
- `drive()`.

It also adds:

```
wheelie()
```

7 Using the Motorcycle Class

```
scooter = Motorcycle("Vespa", 55, 40)  
dirtbike = Motorcycle("KX450F", 80, 25)  
  
scooter.info()  
dirtbike.info()
```

We can also rely on default values:

```
harley = Motorcycle("Harley Davidson")  
  
harley.info()  
print(harley.mpg)  
print(harley.fuel_gal)
```

The object `harley` is created using the default values from the `Motorcycle` constructor.

8 A Generic Motorized Vehicle

We can still instantiate the parent class directly:

```
generic = MotorizedVehicle("People Carrier")  
  
generic.info()  
print(generic.mpg)  
print(generic.fuel_gal)
```

This is useful when the base or intermediate class is meaningful on its own.

9 Inheritance Chain

The inheritance chain in this example is:

`Transportation → MotorizedVehicle → Motorcycle`

Each derived class inherits everything from the classes above it, unless behavior is overridden or hidden.

10 Key Takeaways

- A derived class inherits from a base class.
- Default values make constructors more flexible.
- Derived constructors should initialize the base part of the object.
- In Python, this is often done explicitly with a base-class constructor call.
- Inheritance allows us to reuse common code while adding specialized behavior.

CPSC 250

Inheritance and Polymorphism

Edward Brash

1 Motivation

In many programs, several classes share many of the same features.

If we write those features separately in every class, we duplicate code.

That is inefficient and hard to maintain.

Inheritance gives us a better strategy.

2 The Basic Idea

Inheritance allows us to define a base class containing common information and behavior.

Derived classes then inherit from that base class and add specialized information.

3 Example: Businesses

Suppose we want to model several kinds of businesses.

A general **Business** class might contain:

- name,
- address,
- hours of operation.

A **Restaurant** class might include all of that plus:

- cuisine type,
- rating.

A **CarDealership** class might include all business information plus:

- car makes sold,
- average price by make.

4 Example: Grocery Store Items

Another example is grocery store items.

The base class might be:

```
class Item

    with:

        • name,
        • quantity.
```

Derived classes might include:

- Produce,
- Cereal,
- Meat.

Each derived class has the base features, plus additional specialized features.

5 Base Class: Item

```
class Item:

    def __init__(self):
        self.name = ""
        self.quantity = 0

    def set_name(self, name):
        self.name = name

    def set_quantity(self, quantity):
        self.quantity = quantity

    def display(self):
        print(self.name, self.quantity)
```

6 Derived Class: Produce

```
class Produce(Item):

    def __init__(self):
        Item.__init__(self)
        self.expiration = ""

    def set_expiration(self, expiration):
        self.expiration = expiration

    def get_expiration(self):
        return self.expiration
```

7 Calling the Base Class Constructor

The line:

```
Item.__init__(self)
```

is an explicit call to the base class constructor.

This initializes the inherited variables:

- `name`,
- `quantity`.

This must be done because Python does not automatically call the base constructor in this style.

8 Main Program Example

```
item1 = Item()
item1.set_name("Cereal")
item1.set_quantity(9)
item1.display()

item2 = Produce()
item2.set_name("Apples")
item2.set_quantity(40)
item2.set_expiration("5/5/2024")
item2.display()

print(f"Expires on {item2.get_expiration()}")
```

9 Why This Is Useful

The derived class `Produce` can use inherited methods from `Item`:

- `set_name()`,
- `set_quantity()`,
- `display()`.

It only needs to define what is new:

- expiration date.

10 Polymorphism

Polymorphism means that different classes can be treated in a common way.

For example, if several objects all have a method called:

```
display()
```

then code can call `display()` without needing to know exactly which type of object it is handling.

11 Inheritance Chains

Inheritance can continue over multiple levels.

For example:

Base → DerivedOne → DerivedTwo

Each class adds information to the previous class.

```
class Base:

    def __init__(self):
        self.name = " "

class DerivedOne(Base):

    def __init__(self):
        Base.__init__(self)
        self.address = " "

class DerivedTwo(DerivedOne):

    def __init__(self):
        DerivedOne.__init__(self)
        self.zip_code = " "
```

12 Important Design Point

The notes emphasize that explicitly calling the base class constructor may feel unusual if you come from other languages.

However, it reflects the spirit of inheritance:

The derived class should intentionally initialize the base-class portion of itself.

13 Key Takeaways

- Inheritance reduces duplicated code.
- A base class stores common features.
- A derived class adds specialized features.
- Derived classes can use inherited methods.
- Python often requires explicit base constructor calls.
- Polymorphism allows different objects to be used through common methods.

CPSC 250

Multiple Inheritance and Mixin Classes

Edward Brash

1 Multiple Inheritance

Python allows a class to inherit from more than one base class.

This is called **multiple inheritance**.

For example:

```
class Bat(Winged, Mammal):  
    pass
```

A bat is both winged and a mammal, so it may make sense for it to inherit from both classes.

2 Conceptual Example

Suppose:

```
class Winged:  
  
    def flap_wings(self):  
        print("flap flap")  
  
class Mammal:  
  
    def breathe(self):  
        print("breathing")  
  
    def give_birth(self):  
        print("giving birth")
```

Then:

```
class Bat(Winged, Mammal):  
    pass
```

A Bat object can use methods inherited from both parent classes.

3 Method Name Conflicts

A complication arises if both base classes define a method with the same name.

For example, suppose both parent classes define:

```
blink()
```

Which version does Python use?

4 Method Resolution Order

Python uses the method from the first base class listed, according to its method resolution order.
For example:

```
class Bat(Winged, Mammal):  
    pass
```

If both `Winged` and `Mammal` define `blink()`, Python will search `Winged` first.

5 Mixin Classes

A mixin is a class designed to add a small piece of behavior to another class.
Mixin classes are not usually intended to stand alone.
They are used to compose behavior.

6 Transport Example

Suppose we begin with a base class:

`TransportMode`

Then we want to create:

- a `Truck`,
- a `FlyingCar`.

Both are transport modes, but they move differently.

7 Driving Mixin

```
class DrivingMixin:  
  
    def drive(self, distance):  
        print(f"Driving {distance} miles")
```

8 Flying Mixin

```
class FlyingMixin:  
  
    def fly(self, distance, altitude):  
        print(f"Flying {distance} miles at {altitude} feet")
```

9 Using Mixins

A truck can drive:

```
class Truck(TransportMode, DrivingMixin):  
  
    def go(self, distance):  
        self.drive(distance)
```

A flying car can both fly and drive:

```
class FlyingCar(TransportMode, DrivingMixin, FlyingMixin):  
  
    def go(self, distance):  
        self.fly(distance / 2, self.altitude)  
        self.drive(distance / 2)
```

10 Why Mixins Are Useful

Mixins allow us to keep behavior modular.

Instead of copying the same `drive()` method into several classes, we define it once and inherit it where needed.

This supports:

- code reuse,
- cleaner design,
- better encapsulation,
- simpler maintenance.

11 Using the Objects

The end user should be able to write:

```
s = Truck(...)  
f = FlyingCar(...)  
  
s.go(100)  
f.go(100)
```

and have both objects behave appropriately.

This is another example of polymorphism.

12 Key Takeaways

- Python supports multiple inheritance.
- A class can inherit from more than one parent class.
- If method names conflict, Python follows method resolution order.
- Mixins add focused behavior to classes.

- Mixins help avoid duplicate code.
- Polymorphism allows different objects to use the same interface.

CPSC 250

Plant Information Example

Edward Brash

1 Program Goal

In this example, we use inheritance to build a small plant information program.

We are given:

- a base `Plant` class,
- a derived `Flower` class.

We want to create a list called:

```
my_garden
```

that stores both `Plant` and `Flower` objects.

2 The Base Plant Class

A plant has:

- a name,
- a cost.

```
class Plant:

    def __init__(self, plant_name, plant_cost):
        self.plant_name = plant_name
        self.plant_cost = plant_cost

    def print_info(self):
        print(f"Plant name: {self.plant_name}")
        print(f"Plant cost: {self.plant_cost}")
```

3 The Derived Flower Class

A flower is a plant, but it has additional information:

- whether it is annual,
- flower color.

```
class Flower(Plant):

    def __init__(self, plant_name, plant_cost,
                  is_annual, color_of_flowers):

        Plant.__init__(self, plant_name, plant_cost)

        self.is_annual = is_annual
        self.color_of_flowers = color_of_flowers

    def print_info(self):
        Plant.print_info(self)
        print(f"Annual: {self.is_annual}")
        print(f"Color of flowers: {self.color_of_flowers}")
```

4 The Input Format

The program repeatedly reads user input until the user enters:

```
-1
```

Each input line tells us whether the item is a plant or a flower.

Examples:

```
plant fern 12.99
flower rose 15.99 true red
-1
```

5 Processing the Input

We split the user input into tokens:

```
tokens = user_string.split()
```

Then:

```
type = tokens[0]
name = tokens[1]
cost = tokens[2]
```

If the type is **plant**, we create a **Plant** object.

If the type is **flower**, we create a **Flower** object.

6 Creating Objects

```
if type == "plant":

    my_plant = Plant(name, cost)
    my_garden.append(my_plant)

elif type == "flower":
```

```
is_annual = tokens[3]
color = tokens[4]

my_flower = Flower(name, cost, is_annual, color)
my_garden.append(my_flower)
```

7 The Main Loop

```
my_garden = []

user_string = input()

while user_string != "-1":

    tokens = user_string.split()

    type = tokens[0]
    name = tokens[1]
    cost = tokens[2]

    if type == "plant":

        my_plant = Plant(name, cost)
        my_garden.append(my_plant)

    elif type == "flower":

        is_annual = tokens[3]
        color = tokens[4]

        my_flower = Flower(name, cost, is_annual, color)
        my_garden.append(my_flower)

    user_string = input()
```

8 Printing the List

We write a function:

```
def print_list(my_garden):
```

that loops over every object in the list and calls its `print_info()` method.

```
def print_list(my_garden):

    for i in range(len(my_garden)):

        print(f"Plant {i + 1} Information:")
        my_garden[i].print_info()
        print()
```

9 Polymorphism

This example demonstrates polymorphism.

The list contains both:

- Plant objects,
- Flower objects.

But the loop simply calls:

```
my_garden[i].print_info()
```

Python automatically uses the correct version of `print_info()` depending on the actual object type.

10 Complete Program

```
class Plant:

    def __init__(self, plant_name, plant_cost):
        self.plant_name = plant_name
        self.plant_cost = plant_cost

    def print_info(self):
        print(f"Plant name: {self.plant_name}")
        print(f"Plant cost: {self.plant_cost}")

class Flower(Plant):

    def __init__(self, plant_name, plant_cost,
                  is_annual, color_of_flowers):

        Plant.__init__(self, plant_name, plant_cost)

        self.is_annual = is_annual
        self.color_of_flowers = color_of_flowers

    def print_info(self):
        Plant.print_info(self)
        print(f"Annual: {self.is_annual}")
        print(f"Color of flowers: {self.color_of_flowers}")

def print_list(my_garden):

    for i in range(len(my_garden)):
        print(f"Plant {i + 1} Information:")
        my_garden[i].print_info()
        print()

my_garden = []
```

```

user_string = input()

while user_string != "-1":

    tokens = user_string.split()

    type = tokens[0]
    name = tokens[1]
    cost = tokens[2]

    if type == "plant":
        my_garden.append(Plant(name, cost))

    elif type == "flower":
        is_annual = tokens[3]
        color = tokens[4]
        my_garden.append(Flower(name, cost, is_annual, color))

    user_string = input()

print_list(my_garden)

```

11 Key Takeaways

- A derived class can extend a base class.
- A derived class can override a method from the base class.
- Lists can store objects of related types.
- Polymorphism allows the same method call to behave differently depending on the object.
- Input parsing often begins by splitting strings into tokens.

CPSC 250

Fibonacci Sequence and Algorithm Complexity

Edward Brash

1 The Fibonacci Sequence

The Fibonacci sequence is defined by:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

The sequence begins with:

$$f_0 = 0$$

$$f_1 = 1$$

and every later term is the sum of the previous two:

$$f_n = f_{n-1} + f_{n-2}$$

2 Recursive Fibonacci

The recursive definition naturally suggests a recursive function.

```
def fibonacci(n):  
  
    if n == 0:  
        return 0  
  
    elif n == 1:  
        return 1  
  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

3 Testing the Recursive Function

$$fibonacci(0) = 0$$

$$fibonacci(1) = 1$$

$$fibonacci(2) = fibonacci(1) + fibonacci(0) = 1$$

$$fibonacci(3) = fibonacci(2) + fibonacci(1) = 2$$

The recursive implementation works correctly.

4 The Computational Problem

Although recursive Fibonacci is elegant, it is extremely inefficient.

Why?

Because the function repeatedly recalculates the same values.

For example:

$fibonacci(5)$

requires:

$fibonacci(4)$

and:

$fibonacci(3)$

But:

$fibonacci(4)$

also requires another computation of:

$fibonacci(3)$

and so on.

5 Function Call Explosion

The number of recursive calls grows very rapidly.

Approximate call counts:

n	Number of Calls
0	1
1	1
2	3
3	4
4	7
5	11
6	18
7	29
8	47
9	76

By:

$$n = 30$$

the number of recursive calls is enormous.

6 Complexity of Recursive Fibonacci

The recursive algorithm has approximately exponential complexity:

$$O(e^n)$$

This is extremely bad scaling behavior.
As n increases, runtime grows explosively.

7 Non-Recursive Fibonacci

We can do much better using iteration.

```
def fibonacci(n):  
  
    if n == 0:  
        return 0  
  
    elif n == 1:  
        return 1  
  
    else:  
  
        t1 = 0  
        t2 = 1  
  
        for i in range(2, n + 1):  
  
            t3 = t2 + t1  
  
            t1 = t2  
            t2 = t3  
  
        return t3
```

8 Complexity of Iterative Fibonacci

The iterative version only performs one loop over the data.
Its complexity is:

$$O(n)$$

This is dramatically better than exponential growth.

9 The Golden Ratio

The ratio of successive Fibonacci numbers approaches a constant:

$$\phi \approx 1.618033988\dots$$

called the golden ratio.
Using:

$$f_n = f_{n-1} + f_{n-2}$$

and dividing by f_{n-1} :

$$\frac{f_n}{f_{n-1}} = 1 + \frac{f_{n-2}}{f_{n-1}}$$

As n becomes large:

$$\phi = 1 + \frac{1}{\phi}$$

This gives:

$$\phi^2 - \phi - 1 = 0$$

Using the quadratic formula:

$$\phi = \frac{1 + \sqrt{5}}{2}$$

10 Closed-Form Fibonacci Formula

Binet's formula gives a direct expression for Fibonacci numbers:

$$f_n = \frac{\phi^n - \left(-\frac{1}{\phi}\right)^n}{\sqrt{5}}$$

This suggests a constant-time Fibonacci algorithm.

11 Formula-Based Fibonacci

```
import math

phi = (1.0 + math.sqrt(5.0)) / 2.0

def fibonacci(n):
    return int(
        (phi**n - (-1.0 / phi)**n)
        / math.sqrt(5.0)
    )
```

12 Floating Point Precision Problems

For large n , floating-point precision errors appear.

Why?

Because numbers like:

$$\sqrt{5}$$

and:

ϕ

cannot be represented exactly in binary floating-point form.
Eventually the rounding errors accumulate.

13 Using the Decimal Module

Python's `decimal` module allows arbitrary precision arithmetic.

```
import decimal as dec  
  
dec.getcontext().prec = 50
```

Now calculations can be performed with much higher precision.

14 Summary of Fibonacci Approaches

Method	Complexity	Notes
Recursive	$O(e^n)$	Elegant but slow
Iterative	$O(n)$	Efficient and practical
Formula-Based	$O(1)$	Fast but precision-sensitive

15 Key Takeaways

- Recursive algorithms can be elegant but inefficient.
- Complexity describes how runtime scales with input size.
- Exponential complexity becomes unusable quickly.
- Iterative solutions are often dramatically faster.
- Floating-point precision matters in numerical computing.
- Python's `decimal` module supports high-precision arithmetic.

CPSC 250

Recursion, Binary Search, and Algorithm Efficiency

Edward Brash

1 Recursion

A recursive function is a function that calls itself.

Recursion can be useful when problems naturally break into smaller versions of themselves.

2 A Bad Recursive Example

Consider:

```
def count_down(count):  
    count_down(count - 1)
```

What happens if we call:

```
count_down(3)
```

The function repeatedly calls itself forever:

$$3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow -1 \rightarrow \dots$$

Eventually Python crashes with a recursion error.

3 The Importance of an End Condition

Recursive functions require an end condition.

```
def count_down(count):  
    if count == 0:  
        print("Go!")  
    else:  
        print(count)  
        count_down(count - 1)
```

Now:

count_down(3)

produces:

3, 2, 1, *Go!*

4 Recursion vs Iteration

Although recursion can be elegant, it is often:

- harder to understand,
- harder to debug,
- not more efficient.

One major exception is binary-search-style algorithms.

5 Binary Search Motivation

Suppose we want to find the root of a function:

$$f(x) = 0$$

Assume we know the root lies somewhere between:

$$x_{low}$$

and:

$$x_{high}$$

6 Root Condition

A root exists between two points if:

$$f(x_{low}) \cdot f(x_{high}) < 0$$

This means the function changes sign.

7 The Binary Search Idea

Suppose the root lies between:

$$a$$

and:

$$b$$

We compute the midpoint:

$$c = \frac{a + b}{2}$$

Then determine which half contains the root.

8 Binary Search Algorithm

1. Compute midpoint:

$$c = \frac{a + b}{2}$$

2. Evaluate:

$$f(a)f(c)$$

3. If:

$$f(a)f(c) < 0$$

then the root lies between a and c .

Otherwise it lies between c and b .

4. Repeat.

9 Stopping Condition

The algorithm ends when the interval becomes sufficiently small:

$$|b - a| < \epsilon$$

where:

$$\epsilon$$

is a tiny number.

10 Example Function

```
def f(x):  
    return -4*x + 3
```

This function has a root at:

$$x = 0.75$$

11 Starting Interval

```
a = 0.0  
b = 5.0
```

Check:

```
f(a) * f(b)
```

If the product is negative, a root exists on the interval.

12 Recursive Binary Search Root Finder

```
def find_root(a, b):  
  
    epsilon = 1.0E-5  
  
    if abs(b - a) < epsilon:  
        print("Found root at x =", a)  
  
    else:  
  
        c = (a + b) / 2.0  
  
        if f(a) * f(c) < 0:  
            find_root(a, c)  
  
        else:  
            find_root(c, b)
```

13 Searching Sorted Data

Binary search is also used to search sorted lists.

Suppose we have alphabetically sorted names:

Alice
Charlie
Dave
Frieda
Kallie
Moe
Paul
Sarah
Xavier

We repeatedly check the middle item and eliminate half of the remaining list.

14 Algorithm Efficiency

Algorithm efficiency describes how runtime scales with input size.

This is usually expressed using Big-O notation.

Common examples:

$$O(n)$$

$$O(n^2)$$

$$O(\log n)$$

15 Linear Search

Searching a list one item at a time requires approximately:

$$\frac{n}{2}$$

checks on average.

Complexity:

$$O(n)$$

16 Binary Search Efficiency

Binary search repeatedly cuts the search space in half.

If:

$$n = 2^h$$

then:

$$h = \log_2(n)$$

Therefore binary search complexity is:

$$O(\log n)$$

17 Why Logarithmic Complexity Is Powerful

Suppose:

$$n = 100,000,000$$

A linear algorithm may require roughly:

$$100,000,000$$

operations.

But binary search only requires approximately:

$$\log_2(100,000,000) \approx 27$$

steps.

This is dramatically faster.

18 Key Takeaways

- Recursive functions call themselves.
- Recursive algorithms require stopping conditions.
- Binary search repeatedly divides a problem in half.

- Binary search is extremely efficient.
- Big-O notation describes scaling behavior.
- Logarithmic algorithms are vastly faster than linear algorithms for large datasets.

CPSC 250

Testing Code with Unit Tests and Timers

Edward Brash

1 Motivation

As programs become more complex, it becomes harder to know whether they are correct.

It is also harder to know whether one algorithm is better than another.

Two useful tools help with these questions:

1. unit tests,
2. timing code.

Unit tests help answer:

Does my program do what I think it does?

Timers help answer:

How long does this algorithm take to run?

Together, testing and timing are essential parts of serious software development.

2 Why Unit Tests Matter

Unit testing is not just something used by auto-graders.

Unit tests are widely used in professional software development.

A major use case is **continuous integration**, often abbreviated as CI.

3 Continuous Integration

Imagine a large project hosted on GitHub.

Many people may contribute to the project.

Each person may change one part of the code.

The danger is that a change in one file may accidentally break another part of the project.

Continuous integration systems help by automatically running tests whenever code is updated.

The purpose is to detect broken behavior quickly.

4 Object-Oriented Programming and Testing

Unit tests are especially important in object-oriented programming.

If a class is used by many other parts of a program, then changing one method in that class can accidentally break code elsewhere.

A good set of unit tests helps make sure that the class still behaves correctly after changes.

5 A Simple Example: Circle Class

Suppose we define a simple `Circle` class.

```
class Circle:

    def __init__(self, radius):
        self.radius = radius

    def compute_area(self):
        return 3.14159265 * self.radius ** 2

    def compute_circumference(self):
        return 2.0 * 3.14159265 * self.radius
```

This class has:

- a constructor,
- a method to compute area,
- a method to compute circumference.

6 The unittest Module

Python includes a built-in testing module called:

```
unittest
```

We import it using:

```
import unittest
```

7 Creating a Test Class

A test class should inherit from:

```
unittest.TestCase
```

Example:

```
class TestCircle(unittest.TestCase):
    ...
```

This inheritance gives the test class access to useful testing methods such as:

- `assertEqual()`,

- `assertLess()`,
- `assertGreater()`,
- `assertAlmostEqual()`.

8 Writing a Test Method

Test methods are functions inside the test class.

Important rule:

Test method names must begin with `test_`.

For example:

```
def test_area(self):
    ...
```

The `unittest` framework automatically finds and runs methods whose names begin with `test_`.

9 Testing the Area

A first test checks that a circle of radius zero has area zero.

```
def test_area(self):
    c = Circle(0)
    self.assertEqual(c.compute_area(), 0.0)
```

This test should pass.

10 Testing a Nonzero Area

For a circle of radius 5:

$$A = \pi r^2$$

so:

$$A = \pi(5)^2 = 25\pi \approx 78.5398$$

We can test this:

```
def test_area(self):
    c = Circle(0)
    self.assertEqual(c.compute_area(), 0.0)

    c = Circle(5)
    self.assertAlmostEqual(c.compute_area(), 78.5398, places=4)
```

For floating point values, `assertAlmostEqual()` is often better than `assertEqual()`.

11 Testing for Bad Behavior

Sometimes tests are designed to check that something is *not* true.

For example, the area of a circle with positive radius should not be less than zero.

```
def test_stupid(self):  
  
    c = Circle(5)  
  
    self.assertLess(c.compute_area(), 0.0)
```

This test is expected to fail.

That is useful: it shows that the test framework is actually catching incorrect statements.

12 Running the Tests

At the bottom of the test file, we write:

```
if __name__ == "__main__":  
    unittest.main()
```

This runs the tests when the file is executed directly.

13 Complete Unit Test Example

```
import unittest  
  
class Circle:  
  
    def __init__(self, radius):  
        self.radius = radius  
  
    def compute_area(self):  
        return 3.14159265 * self.radius ** 2  
  
    def compute_circumference(self):  
        return 2.0 * 3.14159265 * self.radius  
  
class TestCircle(unittest.TestCase):  
  
    def test_area(self):  
  
        c = Circle(0)  
        self.assertEqual(c.compute_area(), 0.0)  
  
        c = Circle(5)  
        self.assertAlmostEqual(c.compute_area(), 78.5398, places=4)  
  
    def test_stupid(self):  
  
        c = Circle(5)  
        self.assertLess(c.compute_area(), 0.0)
```

```
if __name__ == "__main__":
    unittest.main()
```

14 Common Assert Methods

Method	Checks That
<code>assertEqual(a, b)</code>	$a == b$
<code>assertNotEqual(a, b)</code>	$a \neq b$
<code>assertTrue(x)</code>	<code>bool(x)</code> is True
<code>assertFalse(x)</code>	<code>bool(x)</code> is False
<code>assertIs(a, b)</code>	a is b
<code>assertIsNot(a, b)</code>	a is not b
<code>assertIsNone(x)</code>	x is None
<code>assertIsNotNone(x)</code>	x is not None
<code>assertIn(a, b)</code>	a in b
<code>assertNotIn(a, b)</code>	a not in b
<code>assertAlmostEqual(a, b)</code>	rounded difference is zero
<code>assertGreater(a, b)</code>	$a > b$
<code>assertGreaterEqual(a, b)</code>	$a \geq b$
<code>assertLess(a, b)</code>	$a < b$
<code>assertLessEqual(a, b)</code>	$a \leq b$

15 Timing Code

Once we have multiple possible algorithms, correctness is not the only question.

We also care about performance.

For example, if two algorithms both produce the correct answer, we may want to know which one is faster.

16 The time Module

Python includes a built-in module called:

```
time
```

This module can be used directly, but it can feel cumbersome.

A useful approach is to create a small wrapper class around the timing functionality.

17 The Timer Wrapper

Suppose we have a file called:

```
timer.py
```

containing a `Timer` class.

Then we can import it using:

```
from timer import Timer
```

18 Using the Timer

The basic pattern is:

```
time1 = Timer()

time1.start()

# do things here

elapsed_time1 = time1.stop()
```

The timer measures how much time passes between:

```
start()
```

and:

```
stop()
```

19 Example: Timing a Loop

```
from timer import Timer

timer = Timer()

timer.start()

total = 0

for i in range(1000000):
    total += i

elapsed_time = timer.stop()

print(f"Elapsed time: {elapsed_time}")
```

20 Why Timing Matters

Timing code helps us compare algorithms in realistic situations.

For example, suppose we have:

- a recursive Fibonacci function,
- an iterative Fibonacci function,
- a formula-based Fibonacci function.

All three may produce the same answer for small n , but their runtime behavior can be dramatically different.

21 Testing vs Timing

Testing and timing answer different questions.

Tool	Main Question
Unit tests	Is the code correct?
Timers	How long does the code take?

Both are important.

Correct but very slow code may not be useful.

Fast but incorrect code is also not useful.

22 Good Workflow

A good development workflow is:

1. Write the code.
2. Write unit tests.
3. Run the tests.
4. Fix errors until the tests pass.
5. Time the code.
6. Compare alternative algorithms.
7. Choose an implementation based on correctness, clarity, and performance.

23 Key Takeaways

- Unit tests check whether code behaves correctly.
- Unit tests are important beyond auto-grading.
- Continuous integration systems use tests to detect broken code automatically.
- Test classes inherit from `unittest.TestCase`.
- Test methods must begin with `test_`.
- `unittest.main()` runs the tests.
- Timers measure algorithm performance.
- Correctness and performance are different but complementary concerns.

Week 4: Plotting, Regression, and Data Analysis

CPSC 250

More on Fibonacci and Complex Numbers

Edward Brash

1 Motivation

The Fibonacci sequence is surprisingly rich.

Previously, we introduced the sequence and discussed several algorithms for computing it.

In this lecture, we return to Fibonacci from a more mathematical point of view.

This gives us a chance to discuss:

- alternate indexing conventions,
- negative Fibonacci numbers,
- Binet's formula,
- non-integer inputs,
- complex numbers in Python,
- and plotting mathematically interesting functions.

2 The Common Fibonacci Convention

A common version of the Fibonacci sequence begins:

$$1, 1, 2, 3, 5, 8, 13, \dots$$

with:

$$f_1 = 1$$

$$f_2 = 1$$

and:

$$f_n = f_{n-1} + f_{n-2}$$

for:

$$n \geq 3$$

This convention uses natural-number indexing.

3 The Zero-Indexed Convention

Earlier, we used a slightly different convention:

$$0, 1, 1, 2, 3, 5, 8, \dots$$

with:

$$f_0 = 0$$

$$f_1 = 1$$

and:

$$f_n = f_{n-1} + f_{n-2}$$

for:

$$n \geq 2$$

This convention is common in programming because indexing often begins at zero.

4 Extending Fibonacci Backward

A natural question is:

Can we extend the Fibonacci sequence to negative indices?

The answer is yes.

Using the recurrence:

$$f_n = f_{n-1} + f_{n-2}$$

we can solve backward.

Since:

$$f_1 = f_0 + f_{-1}$$

and:

$$1 = 0 + f_{-1}$$

we obtain:

$$f_{-1} = 1$$

Next:

$$f_0 = f_{-1} + f_{-2}$$

so:

$$0 = 1 + f_{-2}$$

and therefore:

$$f_{-2} = -1$$

Continuing gives:

$$\dots, 89, -55, 34, -21, 13, -8, 5, -3, 2, -1, 1, 0, 1, 1, 2, 3, 5, \dots$$

The signs alternate for negative indices.

5 Ratio Behavior

For positive indices, the ratio:

$$\frac{f_{n+1}}{f_n}$$

approaches:

$$\phi = \frac{1 + \sqrt{5}}{2}$$

as:

$$n \rightarrow +\infty$$

For negative indices, a related limiting behavior occurs:

$$\frac{f_{n+1}}{f_n} \rightarrow -\frac{1}{\phi}$$

as:

$$n \rightarrow -\infty$$

Thus, the golden ratio appears on the positive side, while its negative reciprocal appears on the negative side.

6 Binet's Formula

Binet's formula is:

$$f_n = \frac{\phi^n - \left(-\frac{1}{\phi}\right)^n}{\sqrt{5}}$$

This formula works beautifully for integer values of n , including negative integers.

7 A Strange Question

Once we have a formula, we can ask a strange but interesting question:

What happens if n is not an integer?

For example:

$$f_{0.5}$$

Using Binet's formula:

$$f_{0.5} = \frac{\phi^{0.5} - \left(-\frac{1}{\phi}\right)^{0.5}}{\sqrt{5}}$$

But:

$$a^{0.5} = \sqrt{a}$$

so this requires:

$$\sqrt{-\frac{1}{\phi}}$$

which is the square root of a negative number.

This leads naturally to complex numbers.

8 Complex Numbers

The imaginary unit is defined by:

$$i = \sqrt{-1}$$

A complex number has the form:

$$z = a + ib$$

where:

- a is the real part,
- b is the imaginary part.

The magnitude of a complex number is:

$$|z| = \sqrt{a^2 + b^2}$$

9 Complex Numbers in Python

Python uses:

j

instead of i for the imaginary unit.

This is common in engineering.

Example:

```
import cmath

a = 4
b = 3

z = complex(a, b)

print(f"The real part of z is {z.real}")
print(f"The imaginary part of z is {z.imag}")
```

This creates:

$$z = 4 + 3j$$

10 Polar Form

The module `cmath` can convert a complex number to polar form.

```
w = cmath.polar(z)

print(f"The size and phase of z are {w}")
```

The result contains:

- magnitude,
- phase angle.

The phase angle is given in radians.

11 Converting Back to Rectangular Form

We can also convert from polar form back to rectangular form:

```
z3 = cmath.rect(w[0], w[1])

print(z3)
```

12 Plotting Fibonacci for Real Values

Now we are ready to ask Python to evaluate Binet's formula for many real values of n , not just integers.

The goal is to make a plot of:

$$f_n$$

versus:

$$n$$

for real values of n between, for example:

$$-30 \leq n \leq 30$$

13 Important Practical Note

For non-integer values of n , Binet's formula may produce complex values.

Therefore, we may need to plot:

- the real part,
- the imaginary part,
- or both.

14 Plotting Plan

A reasonable plan is:

1. Define the Fibonacci function using Binet's formula.
2. Generate a list or array of n -values.
3. Compute the corresponding Fibonacci values.
4. Plot the normal integer Fibonacci values as distinct points.
5. Plot the real and imaginary parts for real-valued input.
6. Draw the horizontal and vertical axes.

15 Matplotlib Axis Lines

Matplotlib provides:

```
plt.axvline(x=0)
plt.axhline(y=0)
```

These draw:

- a vertical line at $x = 0$,
- a horizontal line at $y = 0$.

16 Key Takeaways

- Fibonacci numbers can be indexed in more than one way.
- The sequence can be extended to negative indices.
- Negative-index Fibonacci numbers alternate signs.
- Binet's formula works for integer indices.
- Extending the formula to non-integer inputs naturally introduces complex numbers.
- Python supports complex numbers directly.
- The `cmath` module provides tools for complex arithmetic.

CPSC 250

Basic Plotting of Data

Edward Brash

1 Motivation

We have already seen examples where plotting data is useful.

In this lecture, we develop a base plotting example using `matplotlib`.

The goal is to create a plot with:

1. a title,
2. axis labels,
3. data points with error bars,
4. a fit line,
5. a legend,
6. appropriate axis limits,
7. and possibly logarithmic axes or an error band.

2 Input Data Format

Suppose we have data stored in a CSV file.

The file has columns:

$$x, \quad y, \quad \Delta x, \quad \Delta y$$

where:

- x is the independent variable,
- y is the dependent variable,
- Δx is the uncertainty in x ,
- Δy is the uncertainty in y .

A schematic CSV file might look like:

```
XLabel , YLabel , dXLabel , dYLabel
1.0 , 3.0 , 0.01 , 0.03
2.0 , 3.2 , 0.02 , 0.02
...
```

3 What We Want to Plot

A good data plot should show:

- the data points,
- both x and y error bars,
- the best-fit curve,
- a legend describing the data and fit,
- the x-axis label,
- the y-axis label,
- a title.

4 Reading the Data

We can use Python's `csv` module to read the data.

```
import csv

def read_data(filename):

    x = []
    y = []
    dx = []
    dy = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        headers = next(reader, None)

        for row in reader:

            x.append(float(row[0]))
            y.append(float(row[1]))
            dx.append(float(row[2]))
            dy.append(float(row[3]))

    return headers, x, y, dx, dy
```

5 The Header Row

The line:

```
headers = next(reader, None)
```

reads the first row of the file.

This row contains labels rather than data.

For example:

```
XLabel , YLabel , dXLabel , dYLabel
```

We can use these labels later for axis labels.

6 Creating the Basic Plot

We import:

```
import matplotlib.pyplot as plt
```

Then:

```
filename = "testdata.csv"

headers, xi, yi, dxi, dyi = read_data(filename)

plt.errorbar(xi, yi, xerr=dxi, yerr=dyi, fmt="o")

plt.title("Basic Plotting Example")
plt.xlabel(headers[0])
plt.ylabel(headers[1])

plt.show()
```

7 The errorbar() Function

The function:

```
plt.errorbar()
```

plots points with error bars.

In this example:

```
plt.errorbar(xi, yi, xerr=dxi, yerr=dyi, fmt="o")
```

means:

- use `xi` as x-values,
- use `yi` as y-values,
- use `dxi` as x uncertainties,
- use `dyi` as y uncertainties,
- use circular markers.

8 Axis Limits

Matplotlib will choose default axis limits automatically.

However, sometimes we should set the limits ourselves.

For example:

```
plt.ylim(0, 0)
```

is a special case: Python interprets this as a request to choose the upper limit automatically while forcing the lower limit to zero.

This can be useful when the physical quantity cannot be negative.

9 Which Plot Requirements Are Already Satisfied?

At this stage, we have:

- a title,
- x-axis label,
- y-axis label,
- data points with x and y error bars,
- appropriate automatic axis behavior.

We have not yet added:

- a fit curve,
- a legend,
- logarithmic axes,
- an error band.

10 Fitting

Fitting is a very large topic.

In this course, we use a simple practical approach.

The steps are:

1. Choose a fitting function.
2. Call a fitting routine.
3. Extract the best-fit parameters.
4. Plot the fit.

11 Choosing a Fit Function

Suppose the data appear roughly quadratic.

Then we might choose:

$$y = ax^2 + bx + c$$

In Python:

```
def fitfunction(x, *param):  
    return param[0]*x*x + param[1]*x + param[2]
```

The star in:

```
*param
```

allows the function to accept a variable number of parameters.

Here:

- `param[0]` is a ,
- `param[1]` is b ,
- `param[2]` is c .

12 Calling `curve_fit`

We import:

```
from scipy.optimize import curve_fit
```

Then we call:

```
init_vals = [0.0 for x in range(3)]

popt, pcov = curve_fit(
    fitfunction,
    xi,
    yi,
    p0=init_vals,
    sigma=dyi,
    absolute_sigma=True
)
```

Here:

- `popt` contains the optimal fit parameters,
- `pcov` contains the covariance matrix.

13 Parameter Uncertainties

The uncertainties in the fitted parameters come from the diagonal of the covariance matrix:

```
import numpy as np

perr = np.sqrt(np.diag(pcov))
```

Then:

```
a = popt[0]
b = popt[1]
c = popt[2]

da = perr[0]
db = perr[1]
dc = perr[2]
```

This gives:

$$a \pm \Delta a$$

$$b \pm \Delta b$$

$$c \pm \Delta c$$

14 Plotting the Fit

To plot a smooth curve, create many x-values:

```
xlow = min(xi) - 2
xhigh = max(xi) + 2

xfit = np.linspace(xlow, xhigh, 100)
yfit = fitfunction(xfit, *popt)

plt.plot(xfit, yfit, "--")
```

The line:

```
fitfunction(xfit, *popt)
```

evaluates the fit function using the fitted parameters.

15 Key Takeaways

- CSV files are useful for storing data.
- A good plot includes labels, a title, and a legend.
- Error bars communicate measurement uncertainty.
- `plt.errorbar()` plots data with uncertainties.
- `curve_fit()` performs numerical fitting.
- The covariance matrix provides parameter uncertainties.
- Smooth fit curves are created using many x-values.

CPSC 250

Simple Linear Regression with StatsModels

Edward Brash

1 Motivation

Previously, we used `scipy.optimize.curve_fit()` for fitting data.

In this lecture, we introduce a different approach using:

- `pandas`
- `statsmodels`

The goals are:

1. obtain parameter uncertainties automatically,
2. use a cleaner data structure,
3. simplify regression workflows.

2 Why Pandas?

A `pandas DataFrame` is similar to a spreadsheet.

Each column has a label and stores related data.

This is often easier to work with than raw NumPy matrices.

3 Creating a DataFrame

We first import `pandas`:

```
import pandas as pd
```

Then create a `DataFrame`:

```
df = pd.DataFrame({  
    "x": [5, 10, 15, 20],  
    "y": [5, 12, 20, 27]  
})
```

The keys:

```
"x"  
"y"
```

become the column labels.

4 Accessing Columns

We can print the entire table:

```
print(df)
```

Or access a specific column:

```
print(df["x"])
```

This returns only the x-values.

5 Preparing for Linear Regression

Suppose we want a linear model:

$$y = \beta_0 + \beta_1 x$$

where:

- β_0 is the intercept,
- β_1 is the slope.

6 Importing StatsModels

We typically import:

```
import statsmodels.api as sm
```

7 Extracting the Data

We extract columns from the DataFrame:

```
y = df["y"]  
X = df["x"]
```

Notice:

- uppercase X is used for independent variables,
- lowercase y is used for dependent variables.

8 Adding a Constant Term

We now call:

```
X = sm.add_constant(X)
```

This adds a column of ones.

Why?

Because the model:

$$y = \beta_0 + \beta_1 x$$

contains a constant term.

Without this step, the fit would assume:

$$y = \beta_1 x$$

which forces the line through the origin.

9 Creating the Model

We construct the model using Ordinary Least Squares (OLS):

```
model = sm.OLS(y, X).fit()
```

Important:

- the dependent variable y comes first,
- the independent-variable matrix X comes second.

10 What is OLS?

OLS stands for:

Ordinary Least Squares

This is the standard linear-regression technique.

The idea is to choose parameters that minimize:

$$\sum_i (y_i - y_{\text{fit},i})^2$$

the sum of squared residuals.

11 Viewing the Results

The most useful step is:

```
print(model.summary())
```

This produces a detailed statistical summary including:

- best-fit parameters,
- uncertainties,
- confidence intervals,
- goodness-of-fit measures,
- statistical tests.

12 Advantages of StatsModels

Compared with simpler fitting approaches, `statsmodels` provides:

- cleaner statistical output,
- automatic uncertainty calculations,
- convenient regression tools,
- integration with pandas.

13 Complete Example

```
import pandas as pd
import statsmodels.api as sm

df = pd.DataFrame({
    "x": [5, 10, 15, 20],
    "y": [5, 12, 20, 27]
})

y = df["y"]
X = df["x"]

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

14 Key Takeaways

- Pandas DataFrames organize data using labeled columns.
- StatsModels provides advanced regression tools.
- `sm.add_constant()` adds the intercept term.
- OLS performs standard least-squares regression.
- `model.summary()` gives detailed statistical information.

CPSC 250

Plotting Error Bands

Edward Brash

1 Motivation

When we fit a curve to data, the fit parameters have uncertainties.

A natural question is:

How do uncertainties in the fit parameters translate into uncertainties in the fit itself?

The full statistical treatment is complicated.

However, we can create a useful visualization called an **error band**.

2 The Basic Idea

Suppose our fit parameters are stored in:

- `popt` — optimal fit parameters,
- `pcov` — covariance matrix.

The covariance matrix describes the uncertainties and correlations of the parameters.

We can use this information to generate many random possible fits.

3 Monte Carlo Sampling

The key idea is:

1. randomly generate many possible parameter sets,
2. compute a fit curve for each parameter set,
3. examine the spread of the curves.

4 Generating Random Parameter Sets

We use:

```
ps = np.random.multivariate_normal(  
    popt,  
    pcov,  
    10000  
)
```

This generates:

- 10,000 random parameter sets,
- sampled from a multivariate Gaussian distribution,
- centered on the best-fit parameters.

5 Why a Multivariate Distribution?

Fit parameters are often correlated.

For example:

- increasing one parameter,
- while decreasing another,

might produce similarly good fits.

The covariance matrix captures these correlations.

6 Generating Many Curves

Next:

```
ysample = np.asarray(  
    [  
        fitfunction(xfit, *pi)  
        for pi in ps  
    ]  
)
```

This creates:

- 10,000 fitted curves,
- one for each sampled parameter set.

7 Understanding the Shape of ysample

The array:

```
ysample
```

contains many different possible y-values for each x-position.
Each row corresponds to a different sampled fit.

8 Constructing the Error Band

We now compute percentiles.

Upper boundary:

```
upper = np.percentile(
    ysample,
    84.0,
    axis=0
)
```

Lower boundary:

```
lower = np.percentile(
    ysample,
    16.0,
    axis=0
)
```

9 Why 16% and 84%?

For a Gaussian distribution:

- about 68% of values lie within one standard deviation,
- leaving roughly 16% in each tail.

Therefore:

- the 16th percentile corresponds approximately to -1σ ,
- the 84th percentile corresponds approximately to $+1\sigma$.

10 Plotting the Band

We can shade the region between the boundaries:

```
plt.fill_between(
    xfit,
    lower,
    upper,
    alpha=0.3
)
```

This creates a shaded uncertainty region around the fit.

11 Interpretation

The error band gives a visual estimate of:

- how uncertain the fit is,
- where predictions are more reliable,
- where uncertainties become large.

12 Important Note

This procedure assumes:

- approximately Gaussian parameter uncertainties,
- a reasonable covariance matrix,
- a good fit model.

In professional data analysis, uncertainty estimation can become much more sophisticated.

13 Complete Workflow

Typical steps:

1. Fit the data.
2. Obtain `popt` and `pcov`.
3. Generate many random parameter sets.
4. Compute many fitted curves.
5. Determine percentile boundaries.
6. Plot the shaded region.

14 Key Takeaways

- Fit parameters have uncertainties.
- The covariance matrix describes parameter uncertainty and correlation.
- Monte Carlo sampling can propagate uncertainties into the fitted curve.
- Error bands provide a useful visualization of fit uncertainty.

CPSC 250

Advanced Linear Regression Concepts

Edward Brash

1 Motivation

In earlier lectures, we considered simple linear regression involving one independent variable.

Real-world data analysis is often more complicated.

In this lecture, we discuss:

- multiple regression variables,
- parameter interpretation,
- correlations between variables,
- matrix representations,
- and data structures for regression.

2 Complex Regression Problems

Suppose we want to predict:

y

using several variables:

x_1, x_2, x_3, x_4

For example:

- y : housing price,
- x_1 : square footage,
- x_2 : zip code,
- x_3 : number of bedrooms,
- x_4 : number of bathrooms.

3 The Regression Equation

We seek an equation of the form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4$$

This is linear in the parameters.

4 Why This Is Difficult

Visualizing this problem directly would require a very high-dimensional plot.

For example:

- one dependent variable,
- four independent variables.

This is impossible to visualize completely.

Therefore, regression becomes both:

- a computational problem,
- and an interpretation problem.

5 Parameter Uncertainties

In practice, we determine:

$$\beta_0 \pm \Delta\beta_0$$

$$\beta_1 \pm \Delta\beta_1$$

and so on.

A parameter consistent with zero may indicate that the corresponding variable is not important.

For example:

If the coefficient for number of bedrooms is consistent with zero, housing prices may not strongly depend on that variable.

6 Exploratory Data Analysis

Before fitting, we should first examine the data.

For example:

$$y \text{ vs. } x_1$$

$$y \text{ vs. } x_2$$

and so on.

This helps identify:

- trends,
- bad data,
- outliers,
- nonlinear behavior.

7 Correlations Between Variables

Independent variables may themselves be correlated.

For example:

$$x_2 \text{ vs. } x_3$$

might show a strong trend.

This can produce misleading results.

8 Correlation vs. Causation

An important warning:

Correlation does not imply causation.

Two variables may move together without one causing the other.

Regression analysis must therefore be interpreted carefully.

9 Data Structures for Regression

There are two common choices:

1. NumPy arrays,
2. pandas DataFrames.

10 Matrix Interpretation

Suppose:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

for many data points.

We can write:

$$y_1 = \beta_0 + \beta_1 x_{11} + \beta_2 x_{21}$$

$$y_2 = \beta_0 + \beta_1 x_{12} + \beta_2 x_{22}$$

and so on.

11 Matrix Form

These equations can be rewritten as:

$$\mathbf{y} = X\beta$$

where:

- $\mathbf{y}$ is the vector of dependent values,

- X is the design matrix,
- β is the parameter vector.

12 Linear Algebra Perspective

In principle:

$$\beta = (X^T X)^{-1} X^T \mathbf{y}$$

This is the mathematical foundation of many regression algorithms.

13 Important Practical Point

For most Python regression tools:

- X -data should be a two-dimensional matrix,
- y -data should be a one-dimensional array.

14 Simple NumPy Example

```
import numpy as np

xi = [1, 2, 3, 4, 5]

yi = [1.1, 2.1, 2.9, 3.9, 5.1]

X = np.array(xi).reshape((-1,1))

y = np.array(yi)
```

15 Why Reshape?

The line:

```
reshape((-1,1))
```

converts the array into a column matrix.

This is required because regression libraries expect:

$$N \times M$$

data structures for independent variables.

16 Printing NumPy Arrays

NumPy arrays display differently from Python lists.

For example:

```
print(y)
```

may display:

```
[1.1 2.1 2.9 3.9 5.1]
```

without commas.

17 Data Cleaning

Before fitting, we should:

- plot the data,
- look for missing points,
- identify outliers,
- think about appropriate models.

18 Key Takeaways

- Real regression problems often involve many variables.
- Regression coefficients have uncertainties.
- Correlations between variables can complicate interpretation.
- Regression is fundamentally a linear-algebra problem.
- Most regression libraries require two-dimensional X -data.
- Exploratory plotting is extremely important before fitting.